



Bioinformatics Program

Technical University of Munich

Ludwig-Maximilians-Universität München

Bachelor's Thesis in Bioinformatics

Protein Localization Prediction using Kolmogorov-Arnold Networks

Maximilian Anton Mair





Bioinformatics Program

Technical University of Munich

Ludwig-Maximilians-Universität München

Bachelor's Thesis in Bioinformatics

Protein Localization Prediction using Kolmogorov-Arnold Networks

Vorhersage der Proteinlokalisierung mithilfe von Kolmogorov-Arnold-Netzen

Author:	Maximilian Anton Mair
Supervisors:	Prof. Dr. Burkhard Rost
	Prof. Dr. Julien Gagneur
Advisors:	Dr. Ivan Koludarov
	Dr. Michael Heinzinger
Submitted:	15.10.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Maximilian Anton Mair

Munich, October 15, 2025

Acknowledgments

I would like to extend my deepest gratitude to my supervisors, Dr. Ivan Koludarov and Dr. Michael Heinzinger.

I am particularly thankful for the intellectual freedom and autonomy they granted me throughout this research journey.

While I may not have always sought it out, their incisive guidance and clear-sighted feedback were invaluable whenever I did, and their doors were always open.

I am also deeply grateful for their unwavering patience and personal support throughout my research. Their standard of scientific rigor, their passion and knowledge have been profoundly inspiring. They have set an example of scholarship to which I will aspire for the rest of my career.

On a deeply personal note, I wish to thank my mother. Her unwavering support has been my foundation not just during this thesis, but throughout my entire academic journey. It was not always easy, but she was a constant source of strength.

I am also immensely grateful to my partner, Vera. Her emotional support was my anchor during the most challenging periods of this work, especially in moments when my own resolve faltered. Thank you for carrying me through the doubt and for being my source of light.

Abstract

Accurate prediction of protein subcellular localization is a fundamental challenge in bioinformatics, crucial for understanding protein function and cellular processes. While deep learning models have shown significant promise, the architectural landscape is continually evolving. This thesis presents a rigorous investigation into the efficacy of emerging Kolmogorov-Arnold Networks (KANs) as an alternative to conventional Multi-Layer Perceptron (MLP) architectures for this task.

This thesis develops a comprehensive, modular framework to facilitate a systematic comparison of architectural paradigms, including attention-based, U-Net-based, and a novel positional-based design. The research proceeded in two distinct phases. First, a systematic hyperparameter optimization was conducted to identify promising initial candidates. This process, however, revealed a significant generalization gap, prompting a second, more analytical phase of refinement. This diagnostic stage involved the principled application of architectural regularization, KAN-specific parameter adjustments, and training stabilization techniques to address the specific failure modes identified.

The results demonstrate that this two-phase process was critical for unlocking the potential of the KAN-based models. The top-performing architecture, a refined `AttentionFastKAN` model, achieved a final accuracy of 67.6%, substantially outperforming its initial configuration and the state-of-the-art baseline `LightAttention` model (64.4%). The investigation revealed that the high expressivity of KANs, while enabling rapid convergence, also makes them acutely susceptible to overfitting. A key insight of this work was the identification of the B-spline grid window as a powerful, KAN-specific regularization lever to effectively mitigate this issue and improve generalization.

This work's primary contribution is a deeper understanding of the practical character of KANs in a real-world scientific context. It demonstrates that the architecture's core trade-off between high expressivity and a tendency to overfit can be successfully managed through a synthesis of architectural regularization and the targeted use of the B-spline grid as a KAN-specific control lever. Ultimately, this research provides both a robust framework for comparative architectural analysis and a set of validated strategies that reframe the inquiry from 'if' KANs can be used for complex scientific problems to 'how' they can be made robust and reliable for future research.

Kurzfassung

Die präzise Vorhersage der subzellulären Lokalisierung von Proteinen ist eine grundlegende Herausforderung in der Bioinformatik und entscheidend für das Verständnis der Proteinfunktion und zellulärer Prozesse. Während Deep-Learning-Modelle vielversprechend sind, entwickelt sich die Architekturlandschaft stetig weiter. Diese Arbeit präsentiert eine gründliche Untersuchung der Leistungsfähigkeit der aufkommenden Kolmogorov-Arnold-Netzwerke (KANs) als Alternative zu herkömmlichen Multi-Layer-Perceptron-Architekturen (MLPs) für diese Aufgabe.

Diese Arbeit etabliert einen umfassenden, modularen Rahmen, um einen systematischen Vergleich von Architekturansätzen zu ermöglichen, darunter aufmerksamkeitsbasierte, U-Net-basierte und ein neuartiges positionsbasiertes Design. Die Forschung verlief in zwei Phasen. Zunächst wurde eine systematische Hyperparameteroptimierung durchgeführt, um vielversprechende erste Kandidaten zu identifizieren. Dieser Prozess offenbarte jedoch eine erhebliche Generalisierungslücke, was eine zweite, analytische Verfeinerungsphase veranlasste. Diese Diagnosephase umfasste die prinzipienbasierte Anwendung von Architekturregularisierung, KAN-spezifischen Parameteranpassungen und Techniken zur Trainingsstabilisierung, um die identifizierten Fehlermodi zu adressieren.

Die Ergebnisse zeigen, dass dieser zweiphasige Prozess entscheidend für die Ausschöpfung des Potenzials der KAN-basierten Modelle war. Die leistungsstärkste Architektur, ein verfeinertes `AttentionFastKAN`-Modell, erreichte eine finale Genauigkeit von 67,6% und übertraf damit seine ursprüngliche Konfiguration und das etablierte Referenzmodell `LightAttention` (64,4%) deutlich. Die Untersuchung ergab, dass die hohe Ausdruckskraft von KANs zwar eine schnelle Konvergenz ermöglicht, sie aber auch besonders anfällig für Überanpassung macht. Eine zentrale Erkenntnis dieser Arbeit war die Identifizierung des B-Spline-Gitterfensters als leistungsstarken, KAN-spezifischen Regularisierungshebel, um diesem Problem wirksam entgegenzuwirken und die Generalisierung zu verbessern.

Der primäre Beitrag dieser Arbeit ist ein tieferes Verständnis des praktischen Charakters von KANs in einem realen wissenschaftlichen Kontext. Sie zeigt, dass der zentrale Kompromiss der Architektur zwischen hoher Ausdruckskraft und der Tendenz zur Überanpassung durch eine Synthese aus architektonischer Regularisierung und dem gezielten Einsatz des B-Spline-Gitters als KAN-spezifischer Steuerungshebel erfolgreich bewältigt werden kann. Letztendlich liefert diese Forschung sowohl einen robusten Rahmen für die vergleichende Architekturanalyse als auch eine Reihe validierter Strategien, die die Fragestellung von „ob“ KANs für komplexe wissenschaftliche Probleme verwendet werden können zu „wie“ sie für zukünftige Forschungen robust und zuverlässig gemacht werden können, umformulieren.

Contents

Acknowledgments vii

Abstract ix

Kurzfassung xi

1. Introduction 1
- 1.1. Kolmogorov-Arnold Networks . . . 1
  - 1.1.1. The Kolmogorov-Arnold Representation Theorem . . . 1
  - 1.1.2. From Theorem to Network: Defining Kolmogorov-Arnold Networks (KANs) . . . 1
  - 1.1.3. Key Advantages of KANs over Traditional Networks . . . 2
- 1.2. The Importance of Protein Localization Prediction . . . 3
  - 1.2.1. The Cell as a Highly Organized System . . . 3
  - 1.2.2. Why Prediction is Crucial: Scientific and Practical Advantages . . . 4

2. Methodology 5
- 2.1. Dataset . . . 5
- 2.2. Data Preprocessing and Analysis . . . 5
- 2.3. Protein Sequence Embedding . . . 6
- 2.4. Extensible Model Architecture . . . 7
- 2.5. Modular Feed-Forward Network Framework . . . 7
  - 2.5.1. Rationale and Implementation of the KAN Module . . . 7
- 2.6. A Modular Framework for Architectural Experimentation . . . 8
  - 2.6.1. The `ReducedFFN` Architecture . . . 9
- 2.7. `LightAttention`: A High-Performance Baseline . . . 13
  - 2.7.1. Core Mechanism . . . 13
- 2.8. Model Training and Evaluation . . . 14
  - 2.8.1. Data Collation and Preprocessing . . . 14
  - 2.8.2. Training Procedure and Optimization . . . 15
  - 2.8.3. Performance Metrics and Diagnostics . . . 15
  - 2.8.4. Hyperparameter Optimization . . . 16
  - 2.8.5. Final Model Evaluation and Statistical Validation . . . 17

3. Results and Discussion 19
- 3.1. Performance of Final Models . . . 19
  - 3.1.1. Diagnostic Deep Dive: AttentionFastKAN vs. LightAttention . . . 20
- 3.2. Dataset Characteristics . . . 25
  - 3.2.1. Class Imbalance . . . 25
  - 3.2.2. Sequence Length Distribution . . . 26
- 3.3. Comparative Performance and Model Selection . . . 27
  - 3.3.1. Impact of Reduction Layer Strategies on Performance . . . 28

3.3.2. Analyzing the Performance-to-Size Trade-off 29
3.4. From Optimization to Generalization:
A Refinement Narrative 31
3.4.1. Initial Results and the Overfitting Challenge 31
3.4.2. Taming the KANs: A Journey in Regularization 32
3.4.3. Synergistic Effects and the Final Model Configurations 37

4. Conclusion **43**

A. Code and Data Availability **45**

Code and Data Availability **45**

List of Figures **47**

List of Tables **49**

Bibliography **51**

1. Introduction

1.1. Kolmogorov-Arnold Networks

1.1.1. The Kolmogorov-Arnold Representation Theorem

The theoretical underpinning of modern neural network architectures, like Kolmogorov-Arnold Networks (KANs), is the Kolmogorov-Arnold representation theorem. This profound result in mathematical analysis was first introduced by Andrey Kolmogorov in 1956 and finalized in its more general and powerful form a year later [Kol57], with subsequent contributions by Vladimir Arnold [Arn09].

The theorem states that any multivariate continuous function can be expressed as a finite composition of continuous single-variable functions combined with the simple binary operation of addition.

More formally, for a given multivariate continuous function $f : [0, 1]^n \to \mathbb{R}$, the theorem guarantees the existence of continuous univariate functions $\phi_{q,p} : [0, 1] \to \mathbb{R}$ and $\Phi_q : \mathbb{R} \to \mathbb{R}$, such that [Kol57]:

$$f(x_1, \ldots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{1.1}$$

This representation implies that any complex, high-dimensional function can be decomposed into a series of one-dimensional transformations and summations. In principle, this is highly promising for machine learning, as it suggests that learning a high-dimensional function could be reduced to learning a manageable number of 1D functions.

However, the direct application of the Kolmogorov-Arnold theorem in machine learning has been historically dismissed. A primary obstacle is that the inner and outer functions, $\phi_{q,p}$ and Φ_q, can be non-smooth or even fractal in nature, which makes them exceptionally difficult to approximate or learn with practical algorithms [BG09]. Consequently, the theorem was often regarded as a theoretical curiosity with limited practical utility in the field.

1.1.2. From Theorem to Network: Defining Kolmogorov-Arnold Networks (KANs)

The theoretical elegance of the Kolmogorov-Arnold representation theorem is profound, but it doesn't immediately provide a blueprint for a practical, deep neural network. The theorem guarantees that a function can be represented in a specific two-level structure, but how do we generalize this into a versatile and deep architecture?

The breakthrough, as Liu et al. [Liu+24] propose, comes from re-framing the core components of the theorem in a way that is analogous to modern neural networks.

In a traditional Multi-Layer Perceptron (MLP) [GBC16], a "layer" consists of a linear transformation (learnable weights on the edges) followed by a fixed non-linear activation function applied at the nodes.

KANs fundamentally invert this paradigm. Instead of fixed functions on nodes, KANs place **learnable, univariate activation functions on the edges** of the network. The nodes themselves perform a much simpler operation of just summing up all incoming signals.

This leads to the central question posed by the authors: "What is a KAN layer?" The authors thus define a KAN layer, which connects n_l input nodes to n_{l+1} output nodes, operates via a set of learnable univariate functions $\phi_{l,j,i}$ placed on its edges. The activation for a neuron j in the subsequent layer is then computed as the sum of all incoming function outputs:

$$x_{l+1,j} = \sum_{i=1}^{n_l} \phi_{l,j,i}(x_{l,i})$$

In this equation, $x_{l,i}$ simply represents the activation of the i-th neuron in layer l.

With this definition, the original Kolmogorov-Arnold theorem can be seen as a simple 2-layer KAN with a shape of `[n, 2n+1, 1]`. The power of this new formulation is that we are no longer limited to two layers. A deep KAN is created by simply stacking these layers, where the output vector of one layer becomes the input vector for the next:

$$\mathrm{KAN}(x) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \cdots \circ \Phi_1 \circ \Phi_0)(x)$$

To make the edge functions ϕ learnable, they are parametrized as **B-splines** [Boo78]. This approach is founded on a core principle of approximation theory, which states that any sufficiently smooth function on a bounded interval can be effectively approximated by a linear combination of simple basis functions [Che66]. In a Kolmogorov-Arnold Network, this interval is defined by a grid, and each activation function ϕ is modeled as a weighted sum of these basis splines. The spline coefficients are the trainable parameters that the network learns during back-propagation, allowing it to determine the optimal shape for each activation function on each edge simultaneously [Liu+24].

In essence, while MLPs learn a single, complex non-linear mapping per layer, KANs learn a rich, adaptive combination of many simpler, one-dimensional functions. This entire formulation provides a new and powerful way to approximate complex, high-dimensional functions, as detailed in the foundational paper [Liu+24].

1.1.3. Key Advantages of KANs over Traditional Networks

Kolmogorov-Arnold Networks (KANs) present several compelling theoretical and practical advantages over traditional network architectures like Multi-Layer Perceptrons (MLPs), primarily stemming from their unique structure and its grounding in approximation theory [Liu+24].

A principal advantage lies in their theoretical capacity to mitigate the infamous **Curse of Dimensionality (COD)**. While the Universal Approximation Theorem provides no guarantees on how an MLP's size must scale with input dimension, the approximation error for KANs has been shown to exhibit a convergence rate that is independent of the input dimensionality. This property is a direct consequence of the network's decomposition of a high-dimensional function into a series of one-dimensional spline approximations. By learning and exploiting the intrinsic, low-dimensional compositional structure often present in scientific problems, KANs can achieve a representational efficiency that is not explicitly available to standard MLPs. It is important to note, however, that while the *rate* is dimension-independent, the approximation error *constant* is not, and the network's

ability to overcome the COD is contingent upon the existence of a smooth, compositional representation of the target function [Liu+24].

These structural properties manifest in demonstrably superior **neural scaling laws**. KANs are predicted to have a much faster scaling law, meaning their test error is expected to decrease more rapidly with an increasing number of parameters compared to MLPs. For KANs employing cubic splines, the theoretical scaling exponent is $\alpha = 4$, whereas MLPs are often empirically observed to struggle to saturate much slower scaling laws (e.g., $\alpha \approx 1$). This suggests that KANs can achieve higher accuracy with a more parsimonious parameter budget [Liu+24].

The demonstrated theoretical superiority of KANs in approximation efficiency and scaling provides a strong impetus for empirical validation. The primary objective of this thesis is therefore to investigate whether these advantages translate into tangible performance gains in a real-world, complex bioinformatics application. To this end, KANs are applied to the task of protein subcellular localization, comparing their performance against established, state-of-the-art MLP-based models to test the hypothesis that KANs offer a more powerful and efficient alternative in a practical, scientific setting.

1.2. The Importance of Protein Localization Prediction

1.2.1. The Cell as a Highly Organized System

A defining characteristic of eukaryotic cells is their high degree of internal compartmentalization. The cytoplasm is partitioned by a system of membranes into discrete organelles, each constituting a unique biochemical environment [Ber+18]. This spatial segregation of cellular processes is fundamentally important; it allows for the concentration of specific enzymes and substrates, thereby optimizing the efficiency of metabolic pathways and preventing interference between incompatible or deleterious biochemical reactions. A salient example of this principle is the peroxisome, which sequesters metabolic pathways that generate hydrogen peroxide, a potent and cytotoxic reactive oxygen species. By confining these reactions and co-localizing the catalase enzyme to neutralize the peroxide, the cell is protected from oxidative damage. Similarly, other organelles perform highly specialized functions, such as the nucleus maintaining genomic integrity and regulating gene expression, or the mitochondria executing oxidative phosphorylation for ATP synthesis. This sophisticated functional organization is essential for maintaining cellular homeostasis and executing the complex processes that define eukaryotic life.

The functional integrity of these organelles is contingent upon the precise and regulated trafficking of proteins to their designated subcellular locations. This process, termed protein sorting, ensures that each protein is delivered to the specific compartment where its activity is required, a mechanism that is fundamental to cellular viability [Alb+22]. However, experimentally elucidating the subcellular localization of proteins on a proteome-wide scale remains a significant challenge. Despite advances in experimental techniques like subcellular proteomics and high-resolution imaging, these methods are often labor-intensive, not amenable to high-throughput analysis, and face inherent challenges, particularly with proteins that reside in multiple locations. Consequently, computational methods have emerged as an essential and synergistic approach. This synergy creates a virtuous cycle: advances in high-throughput subcellular proteomics generate the vast, high-quality datasets needed to train more accurate and sophisticated deep learning predictors, providing a rapid means to generate functional hypotheses for uncharacterised proteins [NW22].

1.2.2. Why Prediction is Crucial: Scientific and Practical Advantages

In the post-genomic era, a central challenge in biology lies in translating vast amounts of sequence data into functional biological knowledge. A cornerstone of this endeavour is the prediction of protein subcellular localization, a crucial tool for discovery. A protein's location is intrinsically linked to its biological function, so predicting that a new protein resides in the mitochondria, for instance, immediately generates a testable hypothesis about its role in metabolism or energy production. The field is now pushing beyond organelles to predict localization to sub-organellar compartments—such as the mitochondrial matrix versus its inner membrane—and even to model the dynamic translocation of proteins in response to cellular signals or disease states [NW22].

By identifying where proteins operate, we can map the intricate **cellular mechanisms** that govern life, such as signalling pathways and metabolic cycles, which rely on proteins interacting at specific sites. The field of computational prediction has recently been revolutionized by advances in deep learning, especially techniques adapted from natural language processing. A comprehensive review by Nakai and Wei highlights several facets of this new paradigm [NW22]. By treating protein sequences as a biological language, deep learning models can learn complex hierarchical features indicative of sorting signals. Beyond simple prediction, their interpretability offers profound biological insight; through analysis of 'attention' weights, researchers can identify which specific residues or motifs the model deems critical for determining localization, thereby helping to uncover the molecular mechanisms of protein sorting itself. Moreover, the power of this approach is so great that models can now be used generatively to design entirely novel, functional sorting signals bearing no resemblance to known sequences.

The practical advantages of this predictive power are profound, directly impacting human health, as protein mislocalization is increasingly recognized as a key mechanism in numerous pathologies. A compelling example is found in familial breast and ovarian cancers, where specific mutations in the BRCA1 tumor suppressor gene lead to its mislocalization. Seminal research has demonstrated that these cancer-associated mutations cause the BRCA1 protein to be improperly retained in the cytoplasm, preventing it from reaching the nucleus where it performs its critical function in DNA damage repair [RAH04]. This provides a direct molecular mechanism linking a genetic mutation to the loss of a vital cellular safeguard. Understanding such defects is therefore vital for elucidating disease pathways and accelerating drug development. By identifying a protein's correct operational location—and how that is disrupted in disease—therapies can be designed to be more specific and effective, advancing the paradigm of precision medicine.

2. Methodology

This chapter outlines the methodology employed in this study, from data acquisition and preprocessing to the generation of protein embeddings and the subsequent model training and evaluation protocols.

2.1. Dataset

This research utilizes the dataset from the Light Attention study [Stä+21] to ensure a consistent basis for comparison. The data originates from the DeepLoc dataset [Alm+17], which contains proteins with experimentally verified annotations for ten subcellular locations. Source proteins from UniProt [The17] were filtered to include only eukaryotic, complete sequences longer than 40 amino acids with a single, experimentally confirmed localization (ECO:0000269) [Alm+17].

The final dataset is partitioned as detailed in Table 2.1.

Table 2.1.: Dataset partitions for training, validation, and testing [Stä+21].

Filename	**Purpose**	**# Sequences**
`deeploc_our_train_set.fasta`	Training	9,503
`deeploc_our_val_set.fasta`	Validation	1,158
`setHARD.fasta`	Testing	490

The validation set is redundancy-reduced relative to the training data set to a maximum of 30% pairwise identity (PIDE). For a more rigorous performance benchmark, the `setHARD.fasta` test set was created to evaluate the generalization capabilities. To construct this challenging set, proteins were sourced from SwissProt [The21] and filtered to remove any sequences sharing more than 20% PIDE with any protein in the entire DeepLoc [Alm+17] dataset (both training and test sets) [Stä+21].

2.2. Data Preprocessing and Analysis

The source dataset contains granular subcellular localization annotations, distinguishing, for instance, between membrane-bound versus soluble proteins within the same organelle [Alm+17]. This high specificity results in numerous subclasses with insufficient populations for robust model training. To ameliorate this data sparsity, and in accordance with the original DeepLoc methodology [Alm+17], these granular labels were consolidated into 10 canonical cellular compartments. This aggregation step is crucial for creating a more tractable class distribution, thereby enhancing model stability. The precise mapping is delineated in Table 2.2.

Table 2.2.: Consolidation of granular subcellular annotations into 10 canonical classes.

Final Class Label	Aggregated Original Labels
Cell_membrane	Cell.membrane-M
Cytoplasm	Cytoplasm-S, Cytoplasm-Nucleus-U
Endoplasmic_reticulum	Endoplasmic.reticulum-M, Endoplasmic.reticulum-S, Endoplasmic.reticulum-U
Extracellular	Extracellular-S
Golgi_apparatus	Golgi.apparatus-M, Golgi.apparatus-S, Golgi.apparatus-U
Lysosome_Vacuole	Lysosome/Vacuole-M, Lysosome/Vacuole-S, Lysosome/Vacuole-U
Mitochondrion	Mitochondrion-M, Mitochondrion-S, Mitochondrion-U
Nucleus	Nucleus-M, Nucleus-S, Nucleus-U
Peroxisome	Peroxisome-M, Peroxisome-S, Peroxisome-U
Plastid	Plastid-M, Plastid-S, Plastid-U

Subsequent exploratory data analysis revealed a pronounced class imbalance, a critical finding visualized in the Results (Section 3.2.1). Certain compartments are severely under-represented, namely the **Golgi apparatus** (2.6%), **lysosome/vacuole** (2.3%), and **peroxisome** (1.1%). This marked disparity mandates the implementation of class-aware training strategies and evaluation metrics to ensure the model achieves robust predictive performance across all classes, rather than being biased towards the overrepresented ones [HG09].

2.3. Protein Sequence Embedding

To transform amino acid sequences into a numerical representation amenable to machine learning, each protein was vectorized using the `ProtT5-XL-U50` protein language model [Eln+21]. This model was selected based on its demonstrated superior performance in a cognate study [Stä+21] and generates a 1024-dimensional contextual embedding for each amino acid, resulting in variable-length output tensors corresponding to each protein's sequence.

The embedding generation was performed on a `Google Colab` [Goo25] instance equipped with a `T4 GPU`, adapting the process from the procedure detailed in the `GenerateEmbeddings` notebook [Sen+25] from `ProtSpace` [Sen+24].

The resultant embedding matrices were stored in `HDF5` files [The], a format chosen for its performant I/O operations on large-scale numerical data and its ability to efficiently handle variable-length datasets. `HDF5`'s data chunking architecture facilitates rapid, non-contiguous data retrieval, which is critical for the training pipeline as it enables the direct streaming of batches without necessitating that the entire multi-gigabyte dataset be resident in memory. To ensure stochasticity and mitigate sampling bias, data indices were globally shuffled at the commencement of each training epoch, preceding the streaming process.

2.4. Extensible Model Architecture

To ensure methodological rigor and facilitate reproducible experimentation, a foundational software architecture was engineered. This architecture employs a layered, abstract design to programmatically standardize the entire modeling lifecycle, from development and training to evaluation and hyperparameter optimization. By enforcing a common, agnostic interface for all model instantiations, the framework guarantees that architectural comparisons are scientifically valid and unconfounded by implementation artifacts. Core functionalities, including robust model persistence, version-controlled traceability, and seamless integration with hyperparameter optimization frameworks, were codified at the base level. This design choice was instrumental in both accelerating the exploration of novel architectures and enforcing the strict consistency required for a fair comparative analysis.

2.5. Modular Feed-Forward Network Framework

Within the established experimental framework described above, this thesis conducts a rigorous comparative analysis centered on evaluating the relative merits of two distinct feed-forward network paradigms. To facilitate this analysis, the Feed-Forward Network (FFN) is treated as a modular, interchangeable component, enabling the direct substitution of the novel Kolmogorov-Arnold Network (KAN) [Liu+24] against a canonical Multi-Layer Perceptron (MLP) [GBC16]. In this context, the MLP serves as a well-established baseline, representing the conventional feed-forward architecture and allowing for a focused comparison of these two core component types, independent of the more complex, state-of-the-art models they might be part of.

The structure of the hidden layers for both network types can be specified in two ways: either as a list of exact neuron counts per layer, or as a relative configuration defined by the `num_layers` (the number of hidden layers) and a `relative_size` (the size of each layer relative to the previous one). The MLP module was implemented as a sequence of standard `Linear` layers with `ReLU` activations [NH10]. In contrast, the KAN module leverages the modern Kolmogorov-Arnold Network architecture, whose complexity is further controlled by two hyperparameters for its trainable activation functions: `grid_diff`, which defines the symmetric active input window (`[-x, x]`), and `num_grids`, which sets the resolution of the underlying spline grid.[1]

2.5.1. Rationale and Implementation of the KAN Module

The choice of the underlying KAN library was a critical decision, driven by the need to balance computational performance with architectural fidelity. The original `pykan` library [Liu+24], while foundational, proved too computationally intensive for the scale of this project due to its reliance on B-splines [Boo78].

[1] While the final implementation uses the `faster-kan` library, the module and associated files retain the `FastKAN` name for historical reasons and to avoid extensive code refactoring. The rationale for this specific library choice is detailed in Section 2.5.1.

An initial acceleration was offered by the `fast-kan` library [Li24], which approximates the B-spline basis functions using Gaussian Radial Basis Functions (RBFs) [LB88] [Buh00], defined as:

$$b_i(u) = \exp\left(-\left(\frac{u - u_i}{h}\right)^2\right) \tag{2.1}$$

This approach significantly improves performance, especially on uniform grids.

For this project, however, the even more performant `faster-kan` [Del24] library was ultimately adopted to maximize training throughput. This library further refines the approximation by employing Reflectional Switch Activation Functions (RSWAF), which are defined by the hyperbolic tangent function:

$$b_i(u) = 1 - \left(\tanh\left(\frac{u - u_i}{h}\right)\right)^2 \tag{2.2}$$

Mathematically, this function is equivalent to the squared hyperbolic secant, a smooth *bump* function [FM02]:

$$1 - \tanh^2(x) = \operatorname{sech}^2(x) \tag{2.3}$$

The theoretical validity of using such non-polynomial activation functions for universal approximation is well-established [Les+93].

While the RSWAF basis provides a high-fidelity approximation of the original B-splines with superior computational speed, its reliance on the `tanh` function introduces a potential for steeper gradients compared to the Gaussian RBFs. This characteristic was a suspected contributor to the occasional training instabilities (i.e., exploding gradients) observed during experimentation 3.4.2, a phenomenon that ultimately necessitated the implementation of gradient clipping [PMB13] as a mitigation strategy, which is further discussed in the results. Despite this, the `faster-kan` library provided the optimal balance of training velocity and model fidelity required for the extensive experimental scope of this research.

2.6. A Modular Framework for Architectural Experimentation

This research was driven by two primary objectives. The first was to conduct a comparative analysis by replacing the core MLP-based network of the high-performing `LightAttention` model [Stä+21] with a Kolmogorov-Arnold Network (KAN), thereby evaluating the suitability of KANs for this specific protein localization task. The second, more exploratory goal was to leverage KANs to design a novel model architecture with the potential to surpass the performance of the existing state-of-the-art.

To systematically pursue both objectives, a flexible, two-stage architectural framework was developed. This modular design was essential for enabling direct component substitution (MLP vs. KAN) and for facilitating the rapid prototyping of new models. The framework's design is heavily influenced by the primary technical challenge of the dataset: the high dimensionality of the input embeddings, which form a `1023 x 1024` tensor for each protein.

Consequently, the architecture decomposes the classification problem into two distinct and modular stages, which are described in the following sections.

2.6.1. The `ReducedFFN` Architecture

The initial architectural pattern investigated is termed `ReducedFFN`, a two-stage design that explicitly decouples the task of dimensionality reduction from the final classification. This modular scaffold pairs a dedicated reduction module with a subsequent classification module. The architecture comprises two distinct stages:

1. **Stage 1: Dimensionality Reduction**
   A specialized **`Reduction Layer`** processes the high-dimensional input tensor, transforming it into a smaller, fixed-size representation.

2. **Stage 2: Classification Head**
   The flattened output of the reduction layer is ingested by a **Feed-Forward Network (FFN)**, as described in Section 2.5, which performs the final protein localization classification.

Reduction Layer Strategies

The core of the `ReducedFFN` framework lies in the diverse set of strategies implemented for the dimensionality reduction stage. This exploration is guided by the premise that the method of compressing the protein embedding sequence is a critical determinant of model performance. To this end, a diverse set of reduction strategies was implemented, each representing a different inductive bias for information extraction.

To formalize the description of each layer, we define the input tensor for a single protein sequence as $X \in \mathbb{R}^{L_{in} \times C_{in}}$ accompanied by a boolean mask, where L_{in} is the input sequence length and C_{in} is the number of input channels. Each reduction layer transforms this input into an output tensor $Y \in \mathbb{R}^{L_{out} \times C_{out}}$. These target dimensions, L_{out} and C_{out}, correspond directly to the key hyperparameters `reduced_seq_len` and `reduced_channels`, respectively. The specific transformation from X to Y defines each strategy.

Pooling Layers (`MaxPoolConvLayer`, `AvgPoolConvLayer`)

These non-parametric layers [LeC+02] serve as simple, non-trainable methods for sequence compression. They use 1D adaptive pooling (either maximum or average) to reduce the sequence length to L_{out} while preserving the channel dimension ($C_{out} = C_{in}$). To handle padded sequences correctly, masked positions are either set to negative infinity (for max pooling) or accounted for in a masked average calculation (for average pooling). For an output position j and its corresponding input sub-sequence indices S_j, the operations are formally defined as:

$$\text{Max Pooling: } Y_{j,c} = \max_{i \in S_j, \text{mask}_i = \text{true}} (X_{i,c})$$

$$\text{Avg Pooling: } Y_{j,c} = \frac{\sum_{i \in S_j} X_{i,c} \cdot \mathbb{I}(\text{mask}_i)}{\sum_{i \in S_j} \mathbb{I}(\text{mask}_i) + \epsilon}$$

where $\mathbb{I}(\cdot)$ is the indicator function evaluating to 1 if the mask is true and 0 otherwise, and ϵ is a small constant to ensure numerical stability.

Learnable Linear Projection (`LinearLayer`)

This strategy reduces the sequence length from L_{in} to L_{out} via a learnable linear projection. The transformation is defined by a weight matrix $W \in \mathbb{R}^{L_{out} \times L_{in}}$ and bias vector $b \in \mathbb{R}^{L_{out}}$. These parameters are shared and applied independently across all channels, thus preserving the channel dimension ($C_{out} = C_{in}$), as shown below:

$$Y_{:,c} = WX_{:,c} + b$$

Multi-Head Attention Aggregation (`AttentionLayer`)

This layer utilizes a sophisticated two-stage process built upon the multi-head attention mechanism introduced in the seminal "Attention Is All You Need" paper [Vas+17]. The overall mechanism is composed of two primary components: Scaled Dot-Product Attention and the Multi-Head structure that contains it.

The fundamental building block is Scaled Dot-Product Attention. Its input consists of queries and keys of dimension d_k, and values of dimension d_v. The output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The dot products of the queries and keys are scaled by a factor of $1/\sqrt{d_k}$. As noted by the original authors, this scaling is crucial as it counteracts the effect of large dot product magnitudes, which can push the softmax function into regions with extremely small gradients, thus ensuring stable training.

The Multi-Head Attention mechanism leverages this by performing the attention function in parallel across h different "heads". This allows the model to jointly attend to information from different representation subspaces at different positions. For each head i, the query, key, and value are first linearly projected using dedicated, learnable weight matrices $W_i^Q, W_i^K \in \mathbb{R}^{C_{in} \times d_k}$ and $W_i^V \in \mathbb{R}^{C_{in} \times d_v}$:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The outputs of all parallel heads are then concatenated and passed through a final linear projection $W^O \in \mathbb{R}^{hd_v \times C_{in}}$ to produce the final output of the Multi-Head Attention block:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

This complete ‘MultiHead‘ block is then applied sequentially in two distinct stages:

1. **Stage 1: Self-Attention.** To create a contextually enriched representation of the sequence, the ‘MultiHead‘ function is first applied in a self-attention configuration, where the input sequence X serves as the query, key, and value. This is followed by a residual connection [He+16] and layer normalization [BKH16]:

$$X'_{context} = \text{LayerNorm}(X + \text{MultiHead}(X, X, X))$$

2. **Stage 2: Summarization.** To reduce the sequence length, the `MultiHead` function is applied a second time. In this stage, a set of L_{out} learnable query vectors, $Q' \in \mathbb{R}^{L_{out} \times C_{in}}$, attends to the enriched sequence $X'_{context}$, which serves as both the key and the value. The final output tensor Y is thus:

$$Y = \text{MultiHead}(Q', X'_{context}, X'_{context})$$

This two-stage architecture allows the model to first build a deep contextual understanding of the sequence and then learn a dedicated set of summarization strategies to distill that information into a fixed-size representation. This attention-based summarization approach, using a set of learnable queries, is a key component of modern permutation-invariant architectures such as the Set Transformer [Lee+19].

Gated Positional Convolution (`PositionalWeightedConvLayer`)

To concurrently learn positional importance and extract local feature patterns, a custom layer was designed that integrates a novel position-specific gating mechanism with a standard 1D convolution. This approach also accommodates changes in feature dimensionality ($C_{in} \to C_{out}$). The operation proceeds in three stages:

1. **Positional Gating:**
   First, the input sequence X is modulated by a learnable, data-independent, position-specific gating mechanism. Conceptually, this is related to learnable positional embeddings, a technique popularized by models like BERT [Dev+19], but acts multiplicatively as a gate rather than additively as a bias. This mechanism is conceptually analogous to making the learnable scaling parameter (γ) in a normalization layer, such as LayerNorm [BKH16], position-dependent, thereby learning a unique modulation weight for each feature at each position. The gate is controlled by a parameter matrix $P \in \mathbb{R}^{L_{in} \times C_{in}}$ and is computed as the element-wise sigmoid of these weights, applied via a Hadamard product $\odot$:

   $$X'_{gated} = X \odot \sigma(P/\tau)$$

   Here, $\sigma(\cdot)$ is the sigmoid function and τ is a fixed scalar hyperparameter used to control the sensitivity of the sigmoid gate.

2. **Convolutional Feature Extraction:**
   The gated sequence X'_{gated} is then passed through a 1D convolution, followed by a ReLU activation function [NH10]. The convolution employs a set of learnable filters to extract local patterns and can transform the number of channels from C_{in} to C_{out}:

   $$X''_{conv} = \text{ReLU}(\text{Conv1D}(X'_{gated}))$$

3. **Masked Adaptive Max Pooling:**
   Finally, to reduce the sequence length to L_{out} while correctly handling padded inputs, a masked adaptive max pooling operation is applied [LeC+02]. Prior to pooling, padded positions in the feature map are masked by setting their values to negative infinity ($-\infty$), ensuring they are not selected by the max-pooling operation. The final output Y is then computed for each of the L_{out} output positions by taking the maximum over its corresponding input region S_j:

   $$Y_{j,c} = \max_{i \in S_j, \text{mask}_i = \text{true}} (X''_{conv,i,c})$$

U-Net Inspired Encoder (`UNetLayer`)

This layer leverages the powerful hierarchical feature extraction paradigm of the U-Net architecture [RFB15] to transform the input sequence. It consists of a symmetric encoder-decoder path with skip connections, followed by a final pooling stage to ensure the target output length. The entire operation can be described in three stages:

1. **Encoder Path:** The input sequence X is processed through two levels of encoding. Each level consists of a 1D convolution with a ReLU activation [NH10], followed by a max pooling operation [LeC+02] to downsample the sequence by a factor of two. Crucially, before each pooling operation, padding is handled by applying a mask that sets the values of padded positions to negative infinity, ensuring they are ignored by the max pool. The boolean attention mask itself is also downsampled via pooling to maintain correspondence at each subsequent level.

$$\begin{aligned} X_{conv}^{(1)} &= \text{ReLU}(\text{Conv1D}^{(1)}(X)) \\ X_{pooled}^{(1)} &= \text{MaxPool1D}(\text{Mask}_{-\infty}(X_{conv}^{(1)})) \\ X_{conv}^{(2)} &= \text{ReLU}(\text{Conv1D}^{(2)}(X_{pooled}^{(1)})) \\ X_{pooled}^{(2)} &= \text{MaxPool1D}(\text{Mask}_{-\infty}(X_{conv}^{(2)})) \end{aligned}$$

   where $\text{Mask}_{-\infty}(\cdot)$ denotes the application of the corresponding attention mask to set padded values to $-\infty$.

2. **Decoder Path with Skip Connections:** The decoder path symmetrically upsamples the feature maps using transposed convolutions (`ConvT1D`) [DV16], incorporating high-resolution information from the encoder path via skip connections (concatenation, `Concat`). At each level, the upsampled feature map is concatenated along the channel dimension with the corresponding feature map from the encoder path. This combined tensor is then passed through another convolution.

$$\begin{aligned} U^{(1)} &= \text{ConvT1D}^{(1)}(X_{pooled}^{(2)}) \\ D^{(1)} &= \text{ReLU}\left(\text{Conv1D}^{(dec,1)}\left(\text{Concat}\left(U^{(1)}, X_{conv}^{(2)}\right)\right)\right) \\ U^{(2)} &= \text{ConvT1D}^{(2)}(D^{(1)}) \\ D^{(2)} &= \text{ReLU}\left(\text{Conv1D}^{(dec,2)}\left(\text{Concat}\left(U^{(2)}, X_{conv}^{(1)}\right)\right)\right) \end{aligned}$$

   The output of this path, $D^{(2)}$, is a feature-rich sequence whose length is close to the original L_{in}, and whose channel dimension has been mapped to C_{out}.

3. **Final Adaptive Pooling:** To guarantee the final output length of L_{out}, the sequence $D^{(2)}$ undergoes one final adaptive max pooling operation [LeC+02]. To ensure masking integrity, the original attention mask is resized to match the sequence length of $D^{(2)}$ and then applied by setting padded values to negative infinity. For an output position j and its corresponding input indices S_j, the final output Y is formally computed as:

$$Y_{j,c} = \max_{i \in S_j, \text{mask}_i = \text{true}} (D_{i,c}^{(2)})$$

This hierarchical architecture allows the model to learn and combine both fine-grained local details and abstract, high-level contextual information from the entire protein sequence.

2.7. `LightAttention`: A High-Performance Baseline

To benchmark the custom-developed models against a state-of-the-art architecture from the literature [Stä+21], a faithful implementation of the **`LightAttention`** model was created. As demonstrated by Stark et al., this architecture achieved superior performance over previous methods for protein localization. It provides an efficient, linear-time alternative to the quadratic complexity of standard attention mechanisms [BCB14], making it exceptionally well-suited for processing long protein sequences. The implementation adheres closely to the official code provided by the authors to ensure it serves as a robust and high-performance baseline.

2.7.1. Core Mechanism

The central innovation of `LightAttention` is its use of two parallel 1D convolutions [LeC+02] to generate attention weights and contextual features without quadratic-cost matrix multiplications. For an input sequence embedding $X \in \mathbb{R}^{L \times C_{in}}$, where L is the sequence length and C_{in} is the channel dimension, the mechanism is as follows:

1. **Value and Attention Convolutions:**
   The input X is processed by two independent 1D convolutional layers, whose filter size is determined by the `kernel_size` hyperparameter, to produce a "values" sequence v and an "attention coefficients" sequence e. A dropout layer, controlled by `conv_dropout`, is subsequently applied to the v sequence for regularization.

$$v = \text{Conv1D}_{\text{value}}(X)$$
$$e = \text{Conv1D}_{\text{attention}}(X)$$

2. **Attention Weight Calculation:**
   A `softmax` function is applied along the sequence dimension of the coefficients e to compute the final attention weights α. For each feature channel i and sequence position j:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{l=1}^{L} \exp(e_{i,l})}$$

3. **Context Vector Calculation:**
   A context vector x' is computed as the element-wise product of the attention weights α and the values v, summed over the sequence dimension.

$$x'_i = \sum_{j=1}^{L} \alpha_{i,j} \cdot v_{i,j}$$

4. **Feature Aggregation:**
   To preserve strong, localized signals that might be diminished by the weighted sum, the model also computes a max-pooled [LeC+02] representation $v_{\max}$ directly from the value sequence v.

$$v_{\max,i} = \max_{1 \le j \le L} (v_{i,j})$$

5. **Final Representation:**
   The final sequence representation z is the concatenation of the attention-weighted context vector x' and the max-pooled features $v_{\max}$.

$$z = \text{Concat}(x', v_{\max})$$

To facilitate a direct and controlled comparison of classifier architectures, the final feed-forward network (FFN) of this model was designed as a pluggable component. This setup enabled a rigorous A/B test between two variants:

`LightAttention`:
This version replicates the original classifier from the source paper, which uses a conventional Multi-Layer Perceptron [GBC16] architecture composed of `Linear` → `Dropout` → `ReLU` [NH10] → `BatchNorm1d` [IS15] blocks, where the dropout rate is set by the `ffn_conv_dropout` parameter.

`LightAttentionFastKAN`:
This variant substitutes the conventional MLP classifier with the `FastKAN` architecture 2.5, while all other components of the base model remain identical.

This experimental design allows for a precise evaluation of the impact of replacing a traditional FFN with a Kolmogorov-Arnold Network within an established, high-performance architecture.

2.8. Model Training and Evaluation

Model training and evaluation were conducted within the PyTorch [Pas+19] framework, leveraging automatic mixed-precision training (`torch.amp`) [Mic+17] to enhance computational efficiency and reduce memory usage on the GPU. The process was structured into distinct training and validation loops, enabling systematic model evaluation and checkpointing at the end of each epoch.

2.8.1. Data Collation and Preprocessing

To prepare the variable-length protein sequence embeddings for batch-based training [GBC16], a uniform input sequence length (`in_seq_len`) of 1023 was established for all models. This dimension was determined by a preliminary analysis of the corpus's sequence length distribution, which is visualized in the Results section (see Figure 3.9). The distribution reveals that 1023 is the maximum sequence length produced by the embedding model, making it an optimal choice to minimize information loss from truncation. A custom data collation function, `per_residue_collate_function`, enforces this uniform length by symmetrically truncating or padding both the sequence embeddings and their corresponding boolean padding masks [GBC16]. Shorter sequences are right-padded with zero vectors, while their masks are padded with `False` values. This concurrent generation of data and mask tensors is essential for all downstream model operations, as it allows mechanisms like attention and pooling to distinguish between real data and non-informative padded residues, thereby ensuring computational integrity. This procedure yields a consistent `1023 x 1024` embedding matrix for every protein processed by the data loader.

2.8.2. Training Procedure and Optimization

Model optimization was performed using the `AdamW` optimizer [LH17] and the `CrossEntropyLoss` function [GBC16]. Adhering to the baseline methodology established by Stark et al. [Stä+21], a weight decay (L2 penalty) of 0.0 was used, and the loss function was not class-weighted. The learning rate was initialized to `5e-5` and decayed exponentially by a factor of 0.98 after each epoch to promote stable convergence.

To ensure robust and stable training, several safeguards were implemented. To counteract potential instabilities from exploding gradients, gradient clipping [PMB13] was employed within the mixed-precision loop. Following the backward pass, gradients were first unscaled to their original magnitude before the L2 norm of all model parameters was clipped to a maximum of 1.0. This procedure prevents excessively large parameter updates while maintaining the integrity of the clipping threshold. Furthermore, to handle rare numerical errors, the loss was computed on a per-sample basis, enabling the training loop to detect and discard individual samples within a batch that produced non-finite `NaN` (Not a Number) loss values.[2]

Models were trained for a maximum of **50 epochs** with a **batch size of 28** on a `Google Colab` [Goo25] instance equipped with a `T4 GPU`. To prevent overfitting and ensure the selection of the optimal model checkpoint, an early stopping mechanism [Pre02] was employed with a **patience of 20 epochs**. The decision to halt training was based on the model's composite `performance` score on the validation set (`deeploc_our_val_set.fasta`); if no improvement was observed for 20 consecutive epochs, the training process was terminated. The model instance that achieved the best validation `performance` (see Section 2.8.3) was chosen for subsequent evaluation.

This training configuration, particularly the epoch count, patience and learning rate decay, represents a deliberate deviation from the original Light-Attention paper [Stä+21], which utilized 2500 epochs with a patience of 80 and a constant learning rate. Preliminary experiments indicated that the models developed in this work consistently converged well before the 50-epoch mark. Consequently, the selected parameters were determined to be sufficient to ensure full convergence while significantly improving computational efficiency.

2.8.3. Performance Metrics and Diagnostics

To ensure a robust and comprehensive assessment of model performance, a suite of standard classification metrics was calculated, founded upon the confusion matrix [HTF09]. The primary metrics included accuracy, precision, recall (sensitivity), specificity, and F1-score, which were logged for both the training and validation sets after each epoch to monitor convergence. Given the significant class imbalance in the dataset, these metrics were computed on a per-class basis, and both macro (unweighted) and weighted averages were reported to provide a comprehensive view of performance [HG09].

For the purpose of hyperparameter optimization, which requires a single objective function to maximize, and to act as a more reliable metric, a composite `performance` score was engineered. This score is a weighted sum of the aforementioned metrics, with weights defined heuristically to prioritize metrics robust to class imbalance. Notably, the Area Under the PR Curve (PR AUC) was assigned the highest weight, as it provides a more informative assessment of performance on imbalanced datasets than the ROC AUC [DG06].

[2] This safeguard was initially implemented to handle non-finite loss values observed during preliminary experiments with the pruning functionality of the `pykan` library, a topic revisited in Section 3.4.2.

This composite score served as the primary objective metric guiding the Optuna optimization studies [Aki+19] and for checkpointing the best-performing models against the validation set.

For diagnostic purposes, several visualizations were generated from the best-performing model checkpoint. Normalized confusion matrices, plotted as heatmaps, were used to analyze class-specific error patterns. To assess discriminative capability, both Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves were plotted [DG06; HTF09]. Finally, to visualize prediction certitude, violin plots [HN98; GBC16] depicting the distribution of prediction confidences for each class were generated.

2.8.4. Hyperparameter Optimization

To determine the optimal configuration for each model architecture, a systematic hyperparameter optimization was conducted using the Optuna framework [Aki+19]. The objective was to find the hyperparameters that maximized the composite `performance` score on the validation set. An efficient search of the predefined hyperparameter space was ensured by using a Gaussian Process Sampler (`GPSampler`) [See04]. To maintain computational feasibility, each trial within the study was conducted using an abbreviated training schedule of 20 epochs with an early stopping patience of 14 epochs. The process was designed to be robust, automatically pruning trials that failed due to runtime errors such as out-of-memory conditions.

For each trial, a comprehensive set of performance metrics and model metadata, including memory footprint and specific hyperparameter values, was logged to a structured file. This facilitated a rigorous, multi-stage post-hoc analysis to select the final model configurations based on a multi-criteria evaluation designed to balance raw performance with model efficiency and robustness. Key selection criteria included:

- **Performance and Stability:**
  A primary ranking based on validation accuracy, filtered by a heuristic to exclude models that converged too early (before epoch 6), as this was found to correlate with poor generalization.
- **Performance-to-Size Trade-off:**
  A systematic analysis of the F1-score relative to the model's memory footprint was conducted to select candidates that offered high performance without excessive resource overhead, effectively identifying a point of diminishing returns.
- **Qualitative Assessment:**
  For quantitatively similar candidates, a final qualitative review of their training and validation loss curves and confusion matrices was performed to assess training dynamics and stability.

2.8.5. Final Model Evaluation and Statistical Validation

While the composite `performance` score was instrumental for model selection during optimization, the final assessment of the best-performing models required a more statistically grounded approach. To this end, a bootstrapping procedure was employed for statistical validation [Efr79]. This non-parametric method estimates the sampling distribution of a metric by performing **100 resampling iterations** with replacement from the test set predictions. To ensure a direct and valid comparison with the Light Attention paper [Stä+21], the **mean** and **95% confidence interval** were calculated from the resulting distribution of unweighted micro-averaged accuracy. This provided a statistically sound measure of a model's expected performance and its variance.

3. Results and Discussion

For the sake of reproducibility, all source code and supplementary data underpinning this thesis are openly available in a digital repository, as detailed in Appendix A.

3.1. Performance of Final Models

The primary performance comparison of the final, refined model configurations was conducted on the `setHARD` test set. To ensure a statistically robust evaluation, the final metrics were derived using the bootstrapping procedure detailed in Section 2.8.5. The results for each architecture are presented in Figure 3.1.

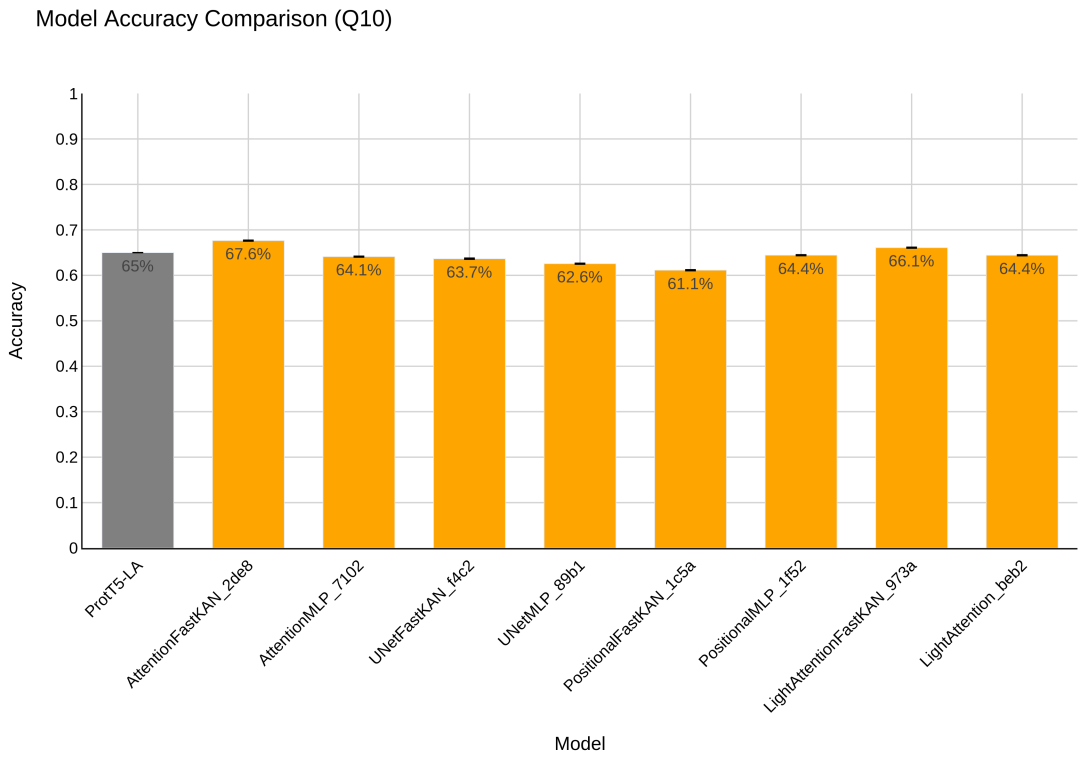


Figure 3.1.: Final model accuracy (mean macro-averaged) comparison on the `setHARD` test set. Error bars represent the 95% confidence interval calculated from 100 bootstrap iterations. Models utilizing a KAN-based classifier are shown in orange, with the original ProtT5-LA (LightAttention) baseline model in gray. The `AttentionFastKAN` model achieved the highest mean accuracy.

A key result from this evaluation is the performance of the **`AttentionFastKAN`** architecture, which achieved the highest mean test accuracy of 67.6%. This surpasses the performance of the original baseline reported by Stark et al. [Stä+21] (ProtT5-LA, 65%)

and our faithfully re-implemented `LightAttention` model (64.4%). This suggests that a KAN-based classifier, when paired with an effective reduction layer, can be a competitive approach for this challenging generalization task.

Observing the direct comparisons within the `ReducedFFN` framework, where only the classifier head was varied, reveals a consistent trend. `AttentionFastKAN` (67.6%) shows a notable performance increase over its direct counterpart, `AttentionMLP` (64.1%). This advantage is also evident within the LightAttention framework, where `LightAttention-FastKAN` (66.1%) surpasses the re-implemented `LightAttention` model (64.4%). A similar, though less pronounced, advantage is observed for `UNetFastKAN` (63.7%) compared to `UNetMLP` (62.6%). The primary exception to this pattern is `PositionalMLP` (64.4%), which performed better than its `PositionalFastKAN` (61.1%) variant. This indicates that the performance benefit of a KAN-based classifier is not universal but may be contingent upon the specific inductive bias of the reduction layer with which it is paired.

3.1.1. Diagnostic Deep Dive: AttentionFastKAN vs. LightAttention

While the superior aggregate accuracy of the `AttentionFastKAN` model is clear, a deeper diagnostic analysis is required to understand the qualitative differences in performance and behaviour compared to the re-implemented `LightAttention` baseline. This analysis focuses on the models' training dynamics, error patterns, and prediction confidences.

Training Dynamics

A comparison of the training and validation curves reveals markedly different learning characteristics for the two architectures (Figures 3.2 and 3.3).

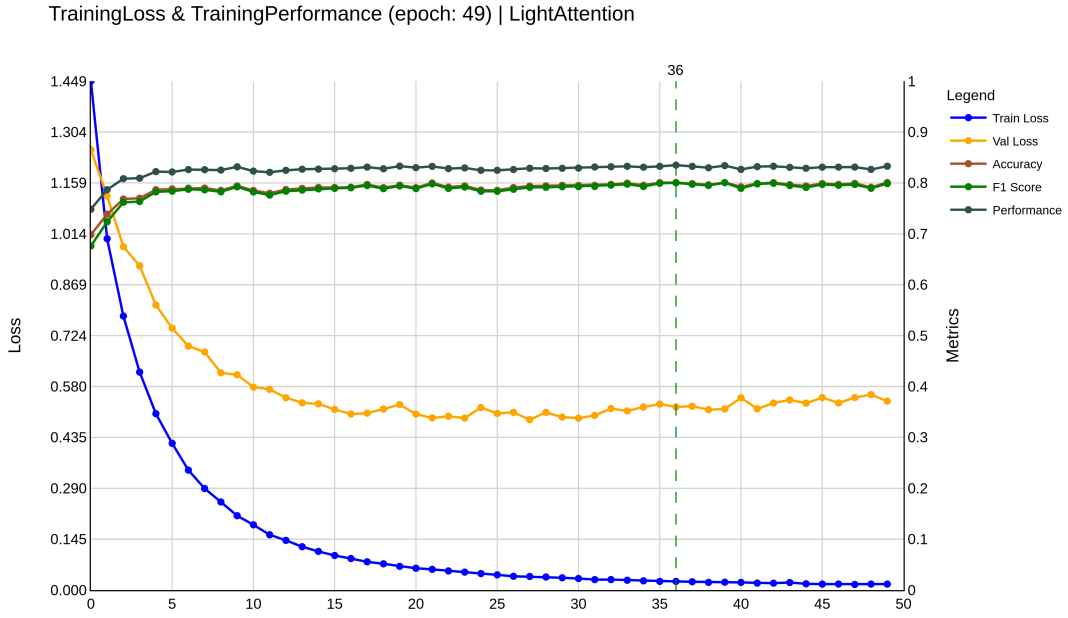


Figure 3.2.: Training and validation curves for the re-implemented `LightAttention` model. Peak performance on the validation set is achieved at epoch 36, after which the validation loss remains stable.

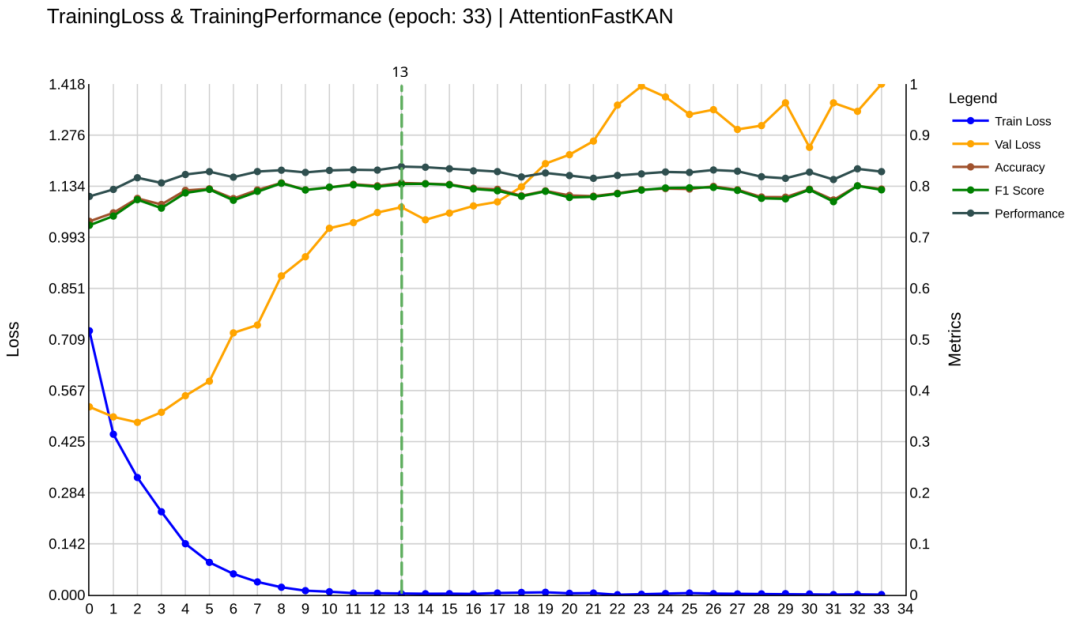


Figure 3.3.: Training and validation curves for the `AttentionFastKAN` model. Note the early rise in validation loss from epoch 3, while peak performance on the validation set is achieved later at epoch 13.

The re-implemented `LightAttention` model exhibits a highly stable training profile. Its training loss decreases steadily without reaching zero, while the validation loss remains exceptionally flat after an initial descent, indicating a strong resistance to overfitting on this dataset. The model's peak `performance` on the validation set was achieved at epoch 36.

In stark contrast, the `AttentionFastKAN` model displays a more aggressive and volatile learning dynamic. While its training loss converges to near-zero with remarkable speed, the validation loss follows a convex trajectory, reaching its minimum as early as epoch 3 before embarking on an almost steady ascent. This divergence is a classic indicator of the model beginning to **overfit**. The apparent contradiction—an increasing validation loss while metrics such as accuracy and F1-score remain stable—provides critical insight into the model's behaviour. Cross-entropy loss is a *soft* metric that penalizes not just incorrect predictions, but also low confidence in correct ones. In contrast, accuracy is a *hard* metric that only measures whether the final predicted class is correct. Despite the early increase in validation loss, these hard metrics continued to improve, with the composite `performance` metric peaking much later at epoch 13.

This tendency towards early specialization can likely be attributed to the high representational capacity of KANs [Liu+24]. Unlike MLPs, where trainable parameters are confined to the weights of linear layers, KANs feature a vast number of trainable coefficients within their B-spline activation functions. This endows them with immense representational capacity, but also renders them more susceptible to memorizing noise from the training data if not carefully regularized.

This leads to a crucial insight: the success of the `AttentionFastKAN` is not merely a product of its architecture but is critically dependent on a well-calibrated early stopping mechanism [Pre02]. Specifically, the protocol checkpointed the model based on the `performance` metric, capturing the point of peak generalization (epoch 13) before the deleterious effects of eroding confidence could further degrade performance. This choice was essential for harnessing the model's full potential and highlights a key characteristic of these KAN-based models: while they learn with remarkable speed, they require careful monitoring with a task-appropriate metric to capture their peak performance before the model's certainty begins to degrade.

Error Analysis via Confusion Matrices

The normalized confusion matrices provide a granular view of each model's classification performance across all 10 classes (Figures 3.4 and 3.5). While both models perform well on several of the well-represented classes, including **nucleus**, **mitochondrion**, and **extracellular**, a closer inspection of their error patterns reveals significant strategic differences, particularly in how they handle the minority classes.

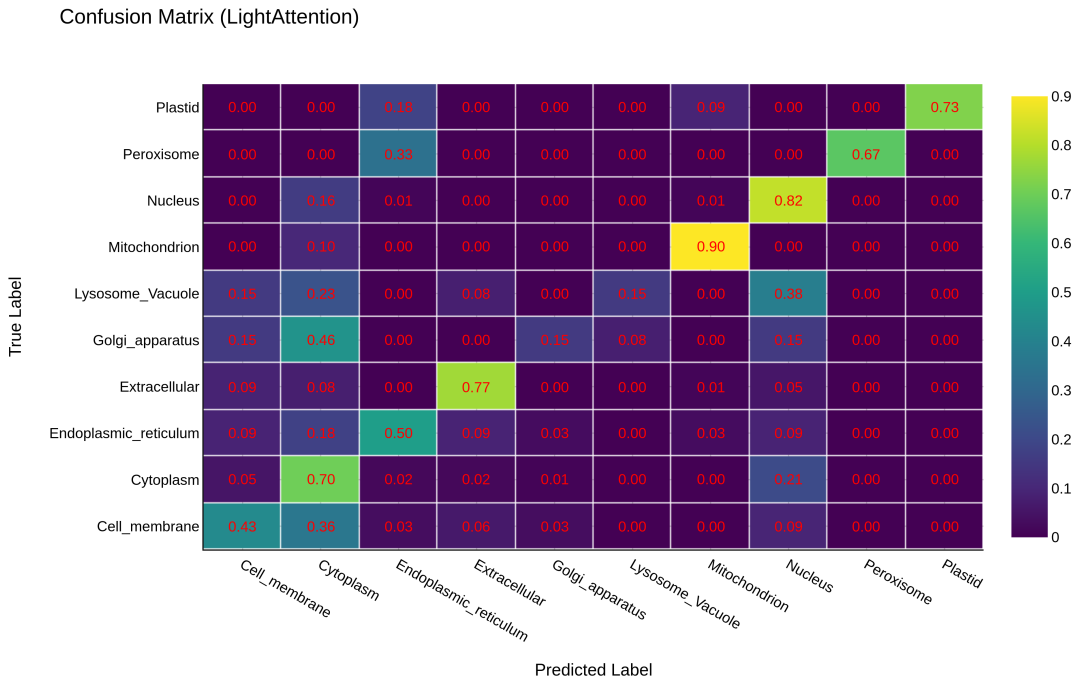


Figure 3.4.: Normalized confusion matrix for the `LightAttention` model on the `setHARD` test set.

A notable discrepancy arises in the classification of **lysosome/vacuole** proteins. The `LightAttention` model misclassified a substantial portion (38%) of these proteins as belonging to the **nucleus**, whereas the `AttentionFastKAN` model reduces this specific error to 15%. A plausible biological interpretation for this common confusion lies in the difficulty of distinguishing a true Nuclear Localization Signal (NLS)—typically a short patch of basic residues—from "cryptic" or "NLS-like" sequences that may exist on other proteins for different structural or functional reasons [Lan+07]. The lower confusion rate of the `AttentionFastKAN` suggests it may have learned a more precise, context-aware representation of this signal.

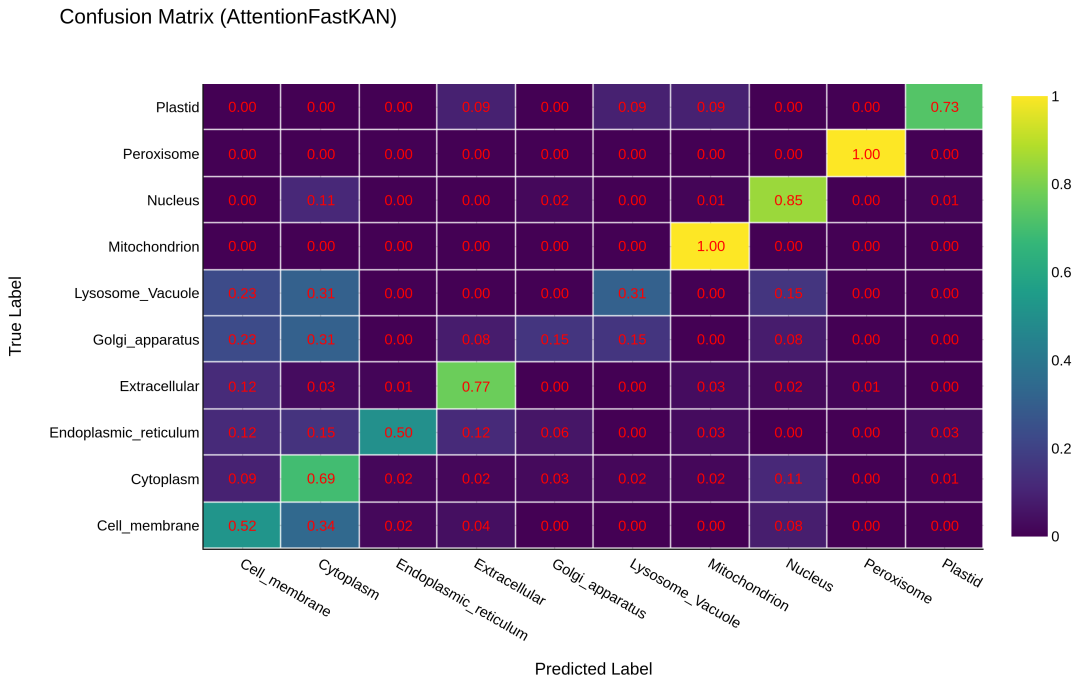


Figure 3.5.: Normalized confusion matrix for the `AttentionFastKAN` model on the `setHARD` test set.

The most striking difference lies in the classification of the rarest classes. The `AttentionFastKAN` model demonstrates a remarkable ability to identify **peroxisome** proteins, achieving a perfect true positive rate (1.00). In contrast, the `LightAttention` model struggles with this class, misclassifying a third (33%) of peroxisome proteins as belonging to the **endoplasmic reticulum**. This specific confusion is biologically plausible, as peroxisomes are known to form from the endoplasmic reticulum, and the import machinery for some peroxisomal proteins involves transient ER localization [Alb+22]. The KAN-based model's ability to perfectly distinguish between these two suggests it may be learning a more subtle or discriminative feature that the MLP-based baseline misses.

Conversely, while both models show some confusion regarding the under-represented **Golgi apparatus**, their error patterns differ. The `AttentionFastKAN` model's primary confusion is with the **cytoplasm** (31%), whereas the `LightAttention` model's errors are more diffuse (41%). A notable shared weakness, however, is the misclassification between **cell membrane** and **cytoplasm**. This is a well-known challenge in protein localization prediction, likely stemming from the large number of transmembrane proteins whose cytosolic domains make their signals inherently ambiguous when compared to purely soluble cytoplasmic proteins [Alb+22].

Overall, the `AttentionFastKAN` matrix shows a pattern of more decisive classifications. This advantage, however, is not universal; performance on several of the most well-represented classes—including **nucleus**, **cytoplasm**, **extracellular**, and **plastid**—was largely equivalent to the baseline model. The key distinction of the `AttentionFastKAN` emerges in its superior handling of the most challenging, low-population classes. This is most evident in its near-perfect classification of **peroxisome** proteins, which suggests that its higher representational capacity may have allowed it to learn the more subtle and specific sequence features associated with the sorting signals of these rare proteins.

Analysis of Prediction Confidence

Further insight into the models' behaviours can be gleaned from the distribution of prediction confidences, visualized as violin plots [HN98; GBC16] in Figures 3.6 and 3.7.

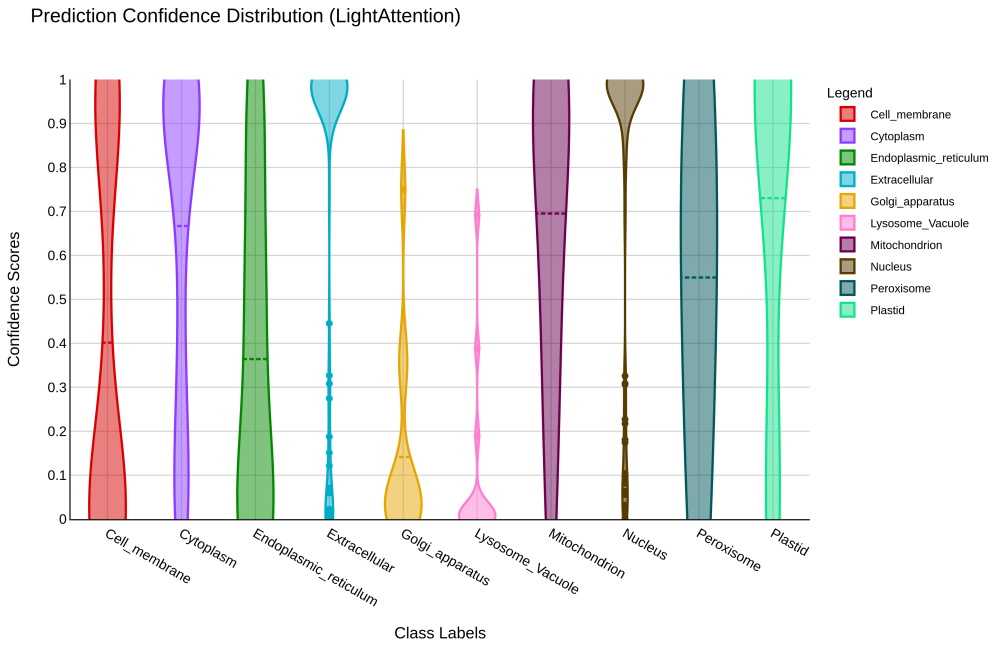


Figure 3.6.: Prediction confidence distribution for the `LightAttention` model. The model displays a reserved confidence profile, with broad distributions for most classes.

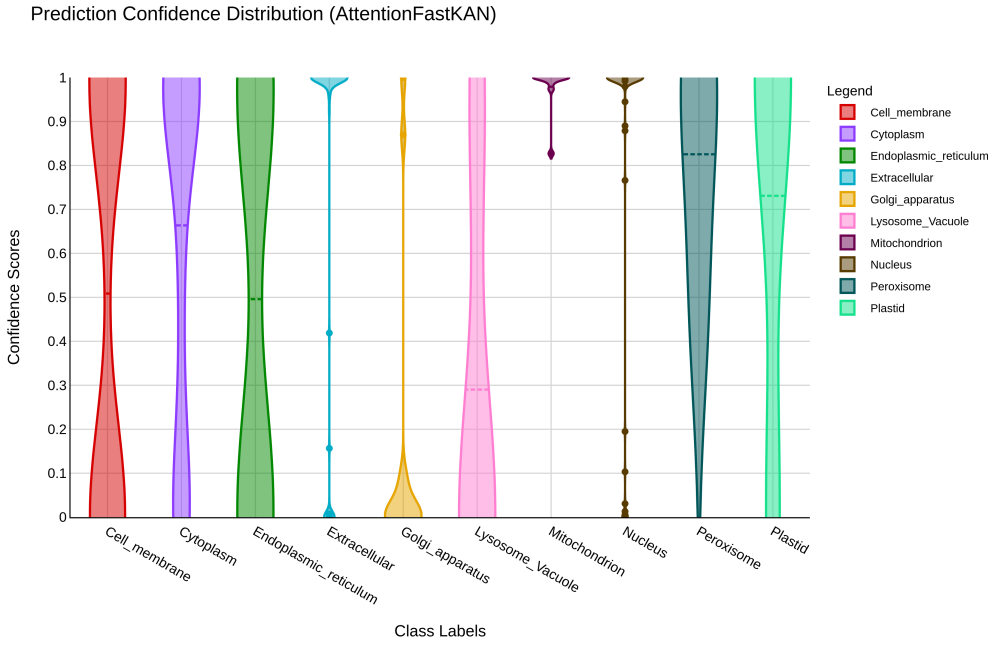


Figure 3.7.: Prediction confidence distribution for the `AttentionFastKAN` model. The model exhibits a more polarized and decisive profile compared to the baseline.

The plots reveal a notable difference in the confidence profiles of the two architectures.

The **Prediction Confidence Violin Plot** [HN98; GBC16] serves as a diagnostic tool for assessing the model's class-specific predictive certainty. For each ground truth class, a distribution is formed by aggregating the confidence scores—the probabilities assigned by the model to that true class—for every sample, regardless of the final prediction outcome. A distribution skewed towards 1.0 thus indicates high confidence and reliable recognition of a class. Conversely, a distribution concentrated near zero reveals systematic uncertainty, in which the model consistently assigns low probability to the correct class, often leading to misclassification.

The plots reveal two distinct predictive strategies. The `LightAttention` model generally exhibits broader, more dispersed confidence distributions across all classes, suggesting a more conservative model. In contrast, the `AttentionFastKAN` model demonstrates a more decisive and specialized character. For several cornerstone organelles—including **mitochondrion**, **nucleus**, and **extracellular**—its confidence distributions are sharply peaked at or near 1.0, indicating exceptionally high conviction. Other notable differences further illustrate these distinct learning patterns. For **peroxisome**, the `LightAttention` model's confidence is centered around the middle (mean approx. 0.50), whereas the `AttentionFastKAN` distribution is strongly skewed towards high confidence (mean approx. 0.82). A different dynamic is observed for the **endoplasmic reticulum**, where the `LightAttention` model shows low confidence (mean approx. 0.37), while the `AttentionFastKAN` adopts a bimodal distribution, suggesting that it may have learned two separate heuristics for this class. Conversely, for the challenging **Golgi apparatus** class, the distribution is heavily skewed towards the lower bound, reflecting consistent uncertainty.

Taken together, these confidence profiles paint a picture of a more specialized learner. The same high representational capacity that allows the `AttentionFastKAN` to achieve near-perfect confidence on classes it understands well also results in a more pronounced and structured pattern of certainty on others. This more decisive predictive signature, particularly its high conviction on challenging minority classes, likely contributes to its superior overall accuracy when compared to the more reserved baseline model.

3.2. Dataset Characteristics

3.2.1. Class Imbalance

A preliminary analysis of the dataset's composition was conducted to inform the modeling strategy and evaluation criteria. This analysis revealed a significant class imbalance across the 10 subcellular localization categories, as visualized in Figure 3.8.

The distribution is heavily skewed, with the two most frequent classes, **nucleus** (28.5%) and **cytoplasm** (19.5%), together accounting for nearly half of the entire dataset. Conversely, several vital organelles are severely under-represented, most notably **peroxisome** (1.1%), **lysosome/vacuole** (2.3%), and **Golgi apparatus** (2.6%).

This pronounced class imbalance is not merely a statistical artifact; it has profound implications that guided the entire methodological and analytical framework of this thesis. From an evaluation perspective, this observation provides the core rationale for the emphasis on imbalance-robust metrics, such as the macro-averaged F1-score and the Area Under the Precision-Recall Curve [DG06]. For model selection during hyperparameter optimization, this principle was applied by engineering a custom, class-weighted composite score, as detailed in the methodology. From a training perspective, the scarcity of training examples for the minority classes presents a significant and well-documented learning

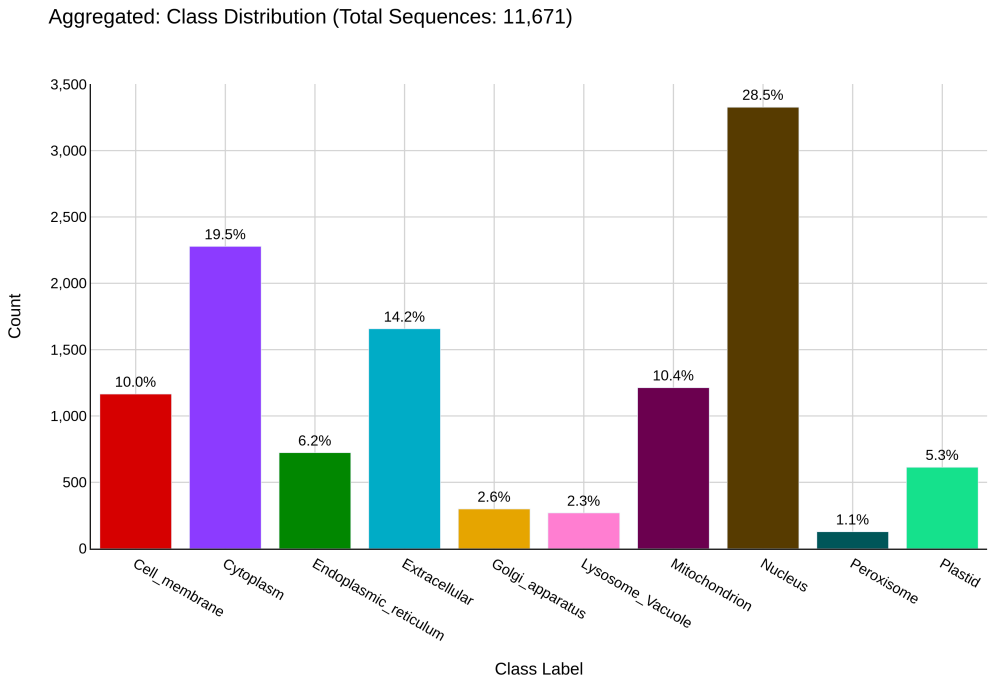


Figure 3.8.: Distribution of the 10 subcellular localization classes across the combined training, validation, and test datasets. The significant class imbalance is evident, with **peroxisome** being the rarest class at 1.1% and **nucleus** the most common at 28.5%.

challenge, often referred to as the problem of learning from imbalanced data [HG09]. Consequently, any successful model must demonstrate that it can effectively learn to distinguish these rare classes despite the skewed distribution. This will be a key focus in the subsequent analysis of model performance, particularly when examining confusion matrices and per-class metrics.

3.2.2. Sequence Length Distribution

Further analysis of the dataset's structural characteristics focused on the distribution of protein sequence lengths, a critical factor influencing model design and data processing. Figure 3.9 presents a log-scaled histogram of these lengths across the entire corpus. A notable feature is the sharp spike at a sequence length of 1023 residues. This is not a biological artifact, but rather a technical one imposed by the `ProtT5` embedding model [Eln+21], which has a maximum context length of 1024 tokens. As the per-residue embeddings used in this study exclude the model's special end-of-sequence token, the effective maximum length for protein residues is 1023. Consequently, longer protein sequences are truncated to this limit, causing them to accumulate at this upper bound. Apart from this truncation-induced spike, the plot reveals a broad and varied distribution for the majority of sequences, with a high density of proteins having lengths between approximately 50 and 600 residues.

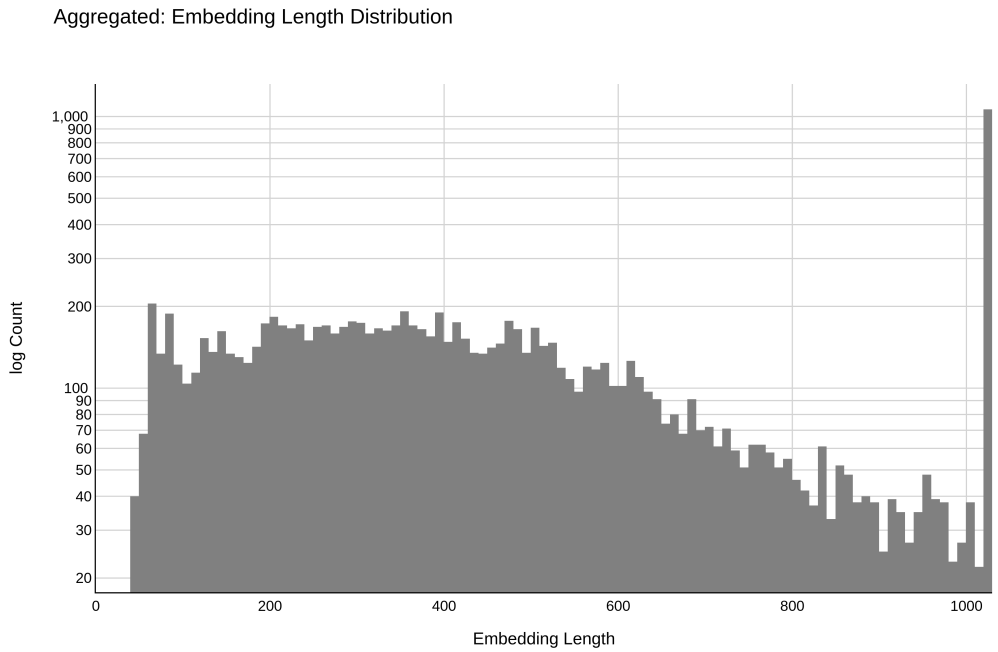


Figure 3.9.: Log-scaled histogram of protein sequence lengths across the aggregated dataset. The prominent spike at 1023 residues corresponds to the effective truncation limit after excluding the end-of-sequence token from the `ProtT5` embedding model. The distribution justifies the selection of 1023 as the uniform sequence length to minimize information loss.

The characteristics of this distribution directly informed and validate the data processing decisions established in the methodology (see Section: 2.8.1). The choice of 1023 as the uniform input sequence length (`in_seq_len`) for all models is a direct consequence of this observation. Selecting this maximum value effectively obviates information loss from truncation, as no sequence in the dataset exceeds this length. Furthermore, the high prevalence of sequences significantly shorter than 1023 underscores the necessity of the padding mechanism implemented in the `per_residue_collate_function`. This strategy, therefore, represents a parsimonious trade-off, balancing the goal of maximum information retention against the computational necessity of uniform tensor dimensions for batch processing.

3.3. Comparative Performance and Model Selection

The hyperparameter optimization yielded a large and diverse cohort of candidate models. To progress from this initial population to a final set of robust architectures, a two-stage selection process was undertaken. The initial stage presents a comparative analysis of the architectural families themselves, identifying the most efficient reduction layer strategies by comparing their aggregate performance. Subsequently, the analysis transitions to a more granular, multi-objective assessment of individual candidates, prioritizing a parsimonious balance between predictive efficacy and computational cost to select the definitive model configurations for this work.

3.3.1. Impact of Reduction Layer Strategies on Performance

Following the extensive hyperparameter optimization process across a wide range of architectures, the initial step in our analysis was to identify the most promising sequence reduction strategies. The goal was to down-select the architectures that demonstrated the highest potential, allowing for a more focused and in-depth comparison in the final evaluation stages. To achieve this, the average validation accuracies of all successful trials from the Optuna study, grouped by their final architecture, were compiled and visualized (see Figure 3.10).

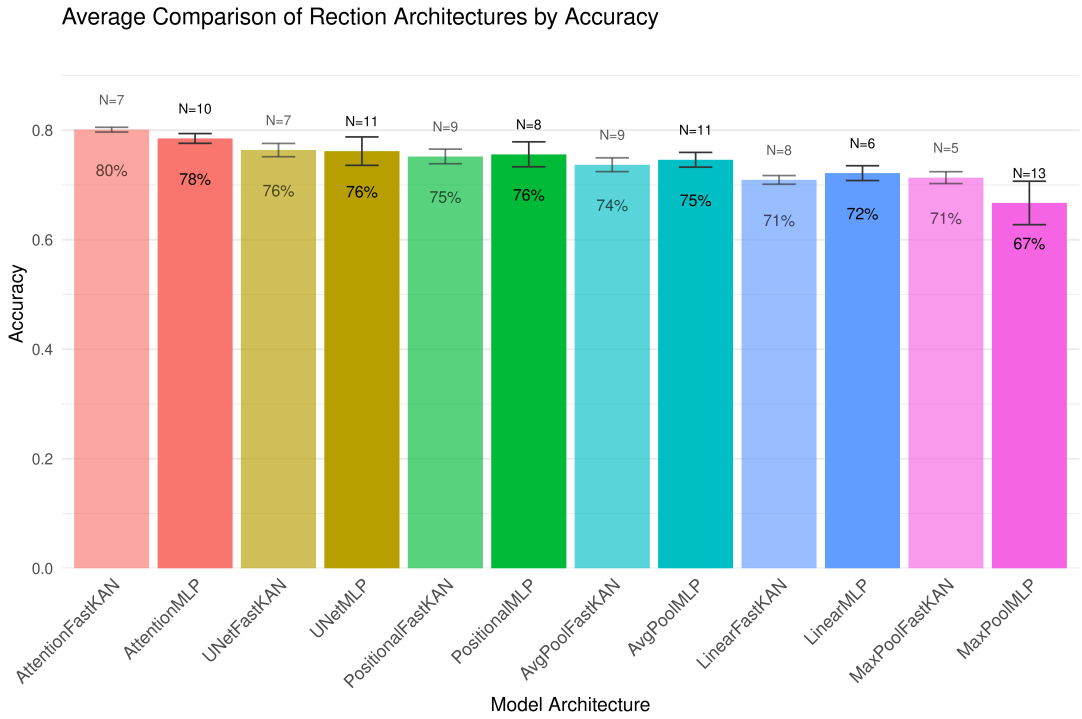


Figure 3.10.: Average **weighted validation accuracy** comparison of different reduction layer architectures from the hyperparameter search. Bars represent the mean accuracy across all successful trials, with error bars indicating the standard deviation. To better distinguish the final feed-forward network, `FastKAN`-based models are rendered with partial transparency, while `MLP`-based models are solid. The `N` value above each bar indicates the number of successful trials included in the average; this number varies due to the automated pruning of failed or prematurely converged trials during the optimization process. The percentage values below show the concrete accuracy score.

The analysis of these aggregated results, as shown in Figure 3.10, revealed a clear performance hierarchy among the different reduction methodologies (see 2.6.1).

The most effective strategy was evidently the attention-based approach [Vas+17]. Models utilizing the `AttentionLayer`, both `AttentionFastKAN` and `AttentionMLP`, achieved the highest average accuracies across all trials, with `AttentionFastKAN` showing a mean accuracy of approximately 80%. This indicates that this mechanism is exceptionally well-suited for identifying and aggregating the most salient features from the ProtT5 embeddings for this classification task.

Forming a second tier of high-performing architectures were those based on the `UNetLayer` and the `PositionalWeightedConvLayer`. Both the KAN and MLP variants of

these architectures demonstrated strong and comparable results, with average accuracies in the range of 75-77%. This suggests that both the U-Net's [RFB15] hierarchical, multi-scale approach and the learnable, position-specific weighting of the convolutional model are powerful strategies for feature extraction to consider.

In contrast, architectures employing simpler, more conventional reduction methods consistently underperformed. Models using `AvgPoolConvLayer`, `MaxPoolConvLayer`, and `LinearLayer` formed the lowest-performing cohort. The `MaxPoolMLP`, for instance, exhibited wide variance and one of the lowest average accuracies, suggesting that simply taking the maximum value across the sequence is an insufficient method for capturing the complex information required for accurate classification.

Based on this clear performance hierarchy, the decision was made to prioritize the most effective reduction strategies for further targeted refinement. The subsequent investigation, therefore, focuses on the `AttentionLayer`, `UNetLayer`, and `PositionalWeightedConvLayer` based architectures, as they demonstrated a clear advantage in extracting meaningful representations from the sequence data.

3.3.2. Analyzing the Performance-to-Size Trade-off

While the initial analysis identified the most promising architectures, selecting the parameter configuration for the final candidates required a more nuanced, multi-criteria evaluation. A key component of this was a systematic analysis of the performance-to-size trade-off, which was guided by the visual inspection of scatter plots mapping each trial's macro F1-score against its corresponding memory footprint (see Figure 3.11).

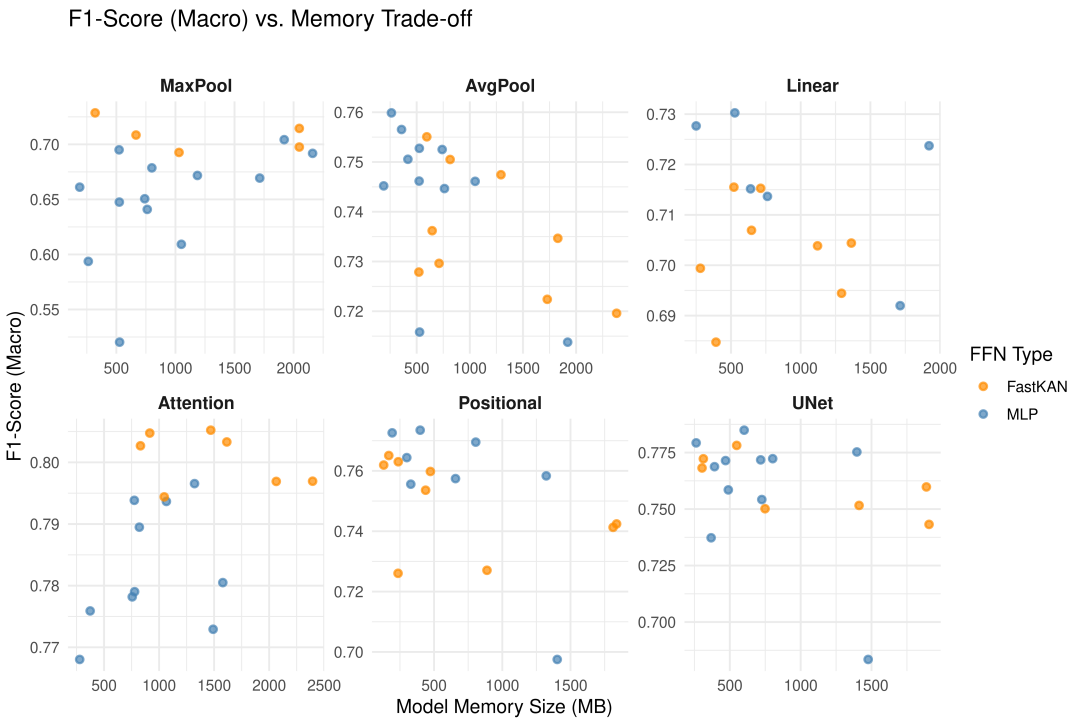


Figure 3.11.: Macro F1-Score versus model memory footprint (in MB) for all successful hyperparameter trials, faceted by base architecture. Each point represents a single trial, colored by its FFN type (`FastKAN`or `MLP`). These plots were used to identify the point of diminishing returns for model size.

The purpose of this visualization was twofold. First, it served as a diagnostic tool to flag potentially inefficient configurations. As seen across multiple facets, such as `AvgPoolConvLayer`- and `PositionalWeightedConvLayer`-based models, there were numerous instances of models with a large memory footprint but a comparatively low F1-score. These candidates could be readily identified and deprioritized.

Second, the plots were used to identify a range of model sizes where performance gains appeared to plateau, the point of diminishing returns. For the most architectures, a visible trend emerges from the sampled data points: the most significant gains occur in the lower-to-mid-memory range. Beyond this point, further increases in memory often yielded only marginal improvements in F1-score, and in some cases, even led to a decrease in performance, likely due to overfitting.

It is important to acknowledge a limitation of this visual analysis: the distribution of data points is determined by the Optuna sampler, which may not have explored all regions of the parameter space with equal density. Furthermore, the selected parameter range may not have been optimal, as subsequent findings revealed that even smaller layer sizes could yield effective models. The apparent clustering of high-performing models in certain memory ranges could be influenced by this sampling bias. This effect is also a natural consequence of the discrete hyperparameters that govern model size, namely the number of hidden layers (`num_layers`) and their relative size (`relative_size`). This effect was likely magnified by the validation set's high sequence similarity to the training set, which provided an imperfect control for generalization and favored larger models that did not perform as well on the test set. However, given that the sampler's goal is to find high-performing regions, the concentration of successful trials in the moderate-size range provides a strong, albeit not exhaustive, signal. The presence of underperforming trials in the very large memory regions further supports the conclusion that simply increasing model size is not a guaranteed path to better performance.

Therefore, this visual analysis directly informed the strategy of targeting candidate model configurations that resided within this empirically identified "sweet spot" of high performance and moderate computational cost. By balancing raw predictive power with efficiency, this methodology ensured that the models selected for the final, intensive evaluation phase were not just the highest-scoring, but also the most practical and generalizable based on the available evidence.

The larger memory footprint of the KAN-based models is an expected consequence of their fundamental design [Liu+24]. Whereas a connection in an MLP is a single scalar weight, a KAN represents each connection as a learnable B-spline function. The number of parameters defining this function is controlled by the `num_grids` hyperparameter. For layers of equivalent input and output dimensions, this results in a multiplicative relationship for the total parameter count (P), which can be approximated as:

$$P_{\text{KAN}} \approx P_{\text{MLP}} \times \texttt{num_grids} \tag{3.1}$$

3.4. From Optimization to Generalization: A Refinement Narrative

The preceding analysis identified a set of prime candidate models, distinguished by their promising performance within the hyperparameter optimization framework. The essential subsequent step was to assess the generalization capabilities of these selected architectures by evaluating them against the test set. This critical evaluation, however, revealed a notable divergence between the models' strong performance on the validation set and their efficacy on previously unseen data. This observation prompted a shift in focus from architectural selection to a deliberate phase of analytical refinement. The narrative that follows details this process of inquiry, through which the sources of this performance gap were investigated to guide the development of more robust and generalizable models.

3.4.1. Initial Results and the Overfitting Challenge

The prime candidate models were subsequently evaluated against the `setHARD` test set to assess their generalization capabilities. For this initial evaluation, models were trained using a **constant learning rate**, and the safeguard of gradient clipping had not yet been implemented.

The resulting accuracies, presented in Figure 3.12, revealed a notable disparity in performance across the different architectures. This discrepancy can be attributed to the deliberate construction of the datasets (as detailed in Section 2.1): the `setHARD` test set's low sequence similarity to the training data presents a more rigorous test of generalization, making it a more challenging benchmark than the validation set.

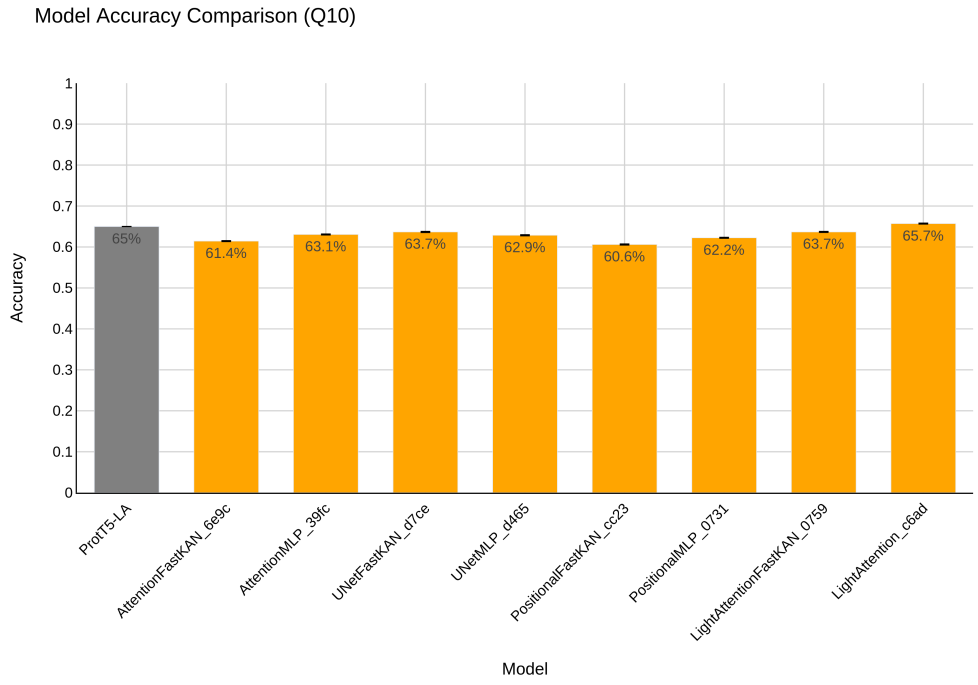


Figure 3.12.: Initial model accuracies on the `setHARD` test set. These first-generation candidates were selected based on hyperparameter optimization against the validation set.

While some architectures appeared competitive, the generalization gap was particularly pronounced in others. For instance, models such as `AttentionFastKAN` (61%) and `PositionalFastKAN` (61%) exhibited markedly lower efficacy on the test set.

This finding was a critical turning point, making it clear that simply optimizing for validation performance was insufficient. A more nuanced approach was required, prompting a phase of principled analysis aimed at diagnosing the factors that limited the generalization of these initial models.

To isolate the performance impact of architectural variations, a common set of foundational parameters was later established across all models, thereby providing a standardized basis for improved comparison.

3.4.2. Taming the KANs: A Journey in Regularization

The observed generalization gap necessitated a focused investigation into regularization strategies to enhance model robustness. Before exploring architecture-specific modifications, a global adjustment was made to the training protocol itself. Based on initial observations that many models converged rapidly, often reaching their lowest validation loss very early, a slight learning rate decay was universally applied. The decay factor was set to 0.98, a modest change intended to promote more stable convergence, particularly for the KAN-based models whose training dynamics were suspected to be more volatile. Unless otherwise specified, all subsequent refinements and experiments discussed in this work were conducted with this learning rate decay in place.

This foundational adjustment was followed by a more targeted inquiry focused on two primary avenues: architectural regularization, to govern overall model complexity, and KAN-specific regularization, to modulate the behaviour of the novel activation functions.

Architectural Regularization: The Power of Information Bottlenecks

The initial hypothesis was that the models derived from the automated search were simply too large, endowing them with excessive capacity to memorize the training data. The first mitigation strategy, therefore, was to manually reduce the size of the FFN classifier head, effectively creating an "information bottleneck" to force the model to learn more compressed and generalizable representations. Inspiration for this approach was drawn from the original `LightAttention` model [Stä+21], which employs a highly constrained classifier, reducing the input dimension from 2048 down to a single hidden layer of just 32 neurons—a reduction to only 1.6% of the original size.

This strategy proved to be highly effective, though its impact was clearly architecture-dependent. The most dramatic performance gains were observed in the `AttentionFastKAN` model, where reducing the classifier from a two-layer configuration at 14% relative size (yielding 64.1% accuracy)[1] to a more stringent single layer at 4% boosted performance to 67.6%.[2] Interestingly, a slight opposite trend was observed for the `AttentionMLP`; a smaller single-layer FFN resulted in a slightly lower accuracy of 63.8%[3] compared to the wider, three-layer configuration that was ultimately chosen (64.1%).[4]

This divergence suggests that for the rich but potentially noisy summary vector produced by the `AttentionLayer`, the combination of a KAN-based classifier with a tight information

[1] `AttentionFastKAN_240a31a5-4230-45da-91ab-4f41c3131412`
[2] `AttentionFastKAN_2de816ef-8c09-4628-8930-52e3fe969738`
[3] `AttentionMLP_2e0e5d90-88c0-4558-8da8-470319aac2d5`
[4] `AttentionMLP_71020308-7a83-46a5-872c-9768ece46f49`

bottleneck is a particularly effective strategy. It is hypothesized that the KAN's expressive activation functions are able to extract salient features even from this highly compressed representation, whereas the MLP counterpart appears to benefit from a larger capacity in its classifier head to achieve its best performance. This finding, however, must be interpreted with the caveat that the hyperparameter space was not exhaustively searched.

In contrast, this aggressive reduction strategy was not found to be as beneficial for the `UNetLayer`-based architectures. The final `UNetFastKAN` model, for instance, required a larger classifier (two layers at 16% relative size) to achieve its peak accuracy of 63.7%.[5] A more aggressive reduction to a single layer at 14% proved detrimental, lowering performance to 61.4%.[6]

A plausible interpretation is that the hierarchical, convolutional nature of the `UNetLayer` acts as a powerful structured feature extractor, producing a "cleaner," more abstract representation. This clean representation can, in turn, be effectively processed by a larger, more expressive classifier without the same degree of overfitting observed in the attention-based models. This finding underscores a key principle of this investigation: effective regularization is not a one-size-fits-all solution, but is contingent on the specific inductive biases of the architectural components involved.

Regularization via Activation Function Refinement

A second, more KAN-specific hypothesis posited that the inherent plasticity of the spline-based activation functions rendered them susceptible to overfitting the idiosyncrasies of the training data. This prompted an inquiry into the regularization potential of the grid parameters themselves, which proved to be a highly effective strategy.

The most consequential adjustment was the reduction of the B-spline grid window, controlled by the `grid_diff` parameter. For most KAN-based models, this value was tightened from an initial 2.0 to 1.5 or 1.0. This change was motivated by the theory that constraining the activations to a more focused input region would diminish the model's sensitivity to high-magnitude outlier activations, which were suspected to be a source of noise.

The effectiveness of this strategy was confirmed during refinement; for example, reducing the `grid_diff` for the `LightAttentionFastKAN` model from 2.0, which yielded an accuracy of 64.0%[7], to a more constrained 1.0 elevated the performance to 65.0%[8].

Concurrently, the resolution of the grid was refined by modestly increasing the number of grid points (`num_grids`), for most models from 7 to 8. While seemingly counter-intuitive, as this marginally increases the parameter count, this change consistently yielded a beneficial regularization effect.

A clear example was observed in the final adjustment of `LightAttentionFastKAN`, where increasing `num_grids` from 8 (accuracy 65.0%)[8] to 10 boosted the final accuracy to 66.1%[9].

This observation might hint that a higher grid resolution within a constrained domain does not invariably lead to overfitting. It could be hypothesized that this affords the model the capacity to learn smoother activation functions, potentially reducing its tendency to capture high-frequency noise from the training data.

[5] `UNetFastKAN_f4c2340a-94be-4547-b5b3-ad2f19a9c633`
[6] `UNetFastKAN_99676e2b-bc68-4f88-871a-8ab8d435fd07`
[7] `LightAttentionFastKAN_05c4b1a3-31c8-4aab-8a8d-3ef344a8fdce`
[8] `LightAttentionFastKAN_c4cd8511-823d-4829-984c-2a8641f543c1`
[9] `LightAttentionFastKAN_973a1bc8-c39d-473c-89f0-1ad2d7ab10f6`

It should be noted that a more exhaustive exploration of these parameters, such as testing even higher `num_grids` values for other models, was not conducted due to project deadlines.

The confluence of these two strategies—constraining the activation domain while refining its internal resolution—proved to be a potent form of regularization. It is pertinent to note that while these parameter settings yielded significant improvements, they were not the result of an exhaustive search. Further tuning was also deliberately curtailed to uphold the integrity of the test set as a final, unbiased arbiter of performance.

A noteworthy exception to the benefits of a constrained grid window was observed in architectures incorporating the `PositionalWeightedConvLayer`. For these models, retaining the larger `grid_diff` of 2.0 proved to be superior, yielding an accuracy of 61.1%[10], whereas a reduction to 1.0 was detrimental to performance (60.1%)[11]. An analysis of this layer's internal mechanics offers a compelling hypothesis for this behaviour.

The `PositionalWeightedConvLayer`, as detailed in Section 2.6.1, is explicitly designed to produce a high-dynamic-range set of activations to encode positional importance. Consequently, constraining the subsequent KAN layer's input domain with a narrow `grid_diff` would prematurely clip these meaningful, high-magnitude signals, preventing the KAN from modeling the very features its upstream layer was engineered to produce. In this context, the regularization is not merely ineffective; it is actively harmful. This finding illustrates that the effectiveness of a regularization strategy is not absolute but is contingent on the statistical properties of its input, reinforcing the theme of co-design between architectural components.

The `PositionalWeightedConvLayer` is distinguished by an explicit, learnable positional weighting mechanism. Crucially, this mechanism includes several design choices that promote a wide dynamic range in its output activations:

- A gradient hook (`lambda grad: grad * 2`) that aggressively amplifies learning for its positional weights.
- A `weights_scalar` that modulates the input to the sigmoid function, preventing premature saturation.
- An explicit clamp (`data.clamp_`) on the weight values themselves acts as a direct form of regularization on the layer's parameters.

It is hypothesized that these features work in concert to produce a rich and varied set of activations intended to encode positional importance. The layer, in effect, utilizes a broader activation space. Consequently, constraining the subsequent KAN layer's `grid_diff` to 1.0 would prematurely clip these meaningful high-magnitude signals, preventing the KAN from effectively modeling the features the `PositionalWeightedConvLayer` was engineered to produce. This layer's internal regularization scheme, therefore, appears to render the external regularization via a narrowed grid window not only unnecessary but detrimental. This hypothesis is further substantiated by the observation that the final `PositionalFastKAN` model also benefited from a reduction in grid resolution (`num_grids` from 7 to 6). It is plausible that because the input signal is already well-structured by the reduction layer, a subsequent KAN function of lower complexity is sufficient for effective modeling.

[10] `PositionalFastKAN_1c5a48ae-76d6-4bac-a42d-ac5a292f6856`
[11] `PositionalFastKAN_b97b8ac3-aa07-4765-88c7-49252ededa74`

Architectural Refinement: A Two-Stage Attention Mechanism

A pivotal architectural refinement was implemented within the `AttentionLayer` to augment its capacity for capturing complex, long-range dependencies inherent in protein sequences. The antecedent design employed a single cross-attention step, wherein a set of learnable query vectors attended directly to the input residue embeddings. While computationally efficient, this monolithic approach precludes any preliminary exchange of information between the input residues themselves. Consequently, the summarization stage operates on context-naive representations, limiting its ability to capture intricate sequence motifs.

To surmount this limitation, the layer was re-conceptualized as a sequential, two-stage process. The first stage introduces a self-attention mechanism, a foundational component of the Transformer architecture [Vas+17]. In this stage, the input sequence attends to itself, allowing each residue's representation to be updated through the integration of information across the entire sequence, thereby creating a context-aware intermediate representation. This operation, followed by a residual connection [He+16] and layer normalization [BKH16], produces a contextually-enriched intermediate representation.

Only subsequent to this enrichment stage does the second stage commence. Here, the set of learnable summary queries attends to the now enriched sequence. This method of employing attention with a dedicated set of learnable queries to produce a fixed-size output is a hallmark of permutation-invariant architectures, most notably the Set Transformer [Lee+19].

This revised two-stage architecture establishes a more powerful inductive bias. It enables the model to first construct a comprehensive understanding of inter-residue dependencies before distilling this richer information into a final, compact representation for downstream processing. The empirical results confirm that this architectural enhancement was crucial for improving the model's predictive acuity regarding subtle protein sorting signals.

The Gradient Instability Challenge

Beyond the primary challenge of generalization, a secondary issue concerning training stability was identified in the KAN-based models. A representative example of this phenomenon is depicted in Figure 3.13, which illustrates the training dynamics of an explanatory `LightAttentionFastKAN` model[12].

While the training loss generally decreases, the process is punctuated by severe, intermittent spikes. These events are a classic symptom of exploding gradients, a form of numerical instability that can derail the optimization process. The origin of this behaviour was traced to the mathematical formulation of the activation functions within the `faster-kan` library [Del24]. The activation function itself, which is based on a $1 - \tanh^2(x)$ term (as detailed in Section 2.5.1), has regions with steep gradients that are prone to producing exceptionally large derivative values. This inherent vulnerability was likely exacerbated by the use of mixed precision training, whose half-precision floating-point format has a much smaller dynamic range and is therefore less tolerant of sudden gradient spikes [Mic+17].

While reverting to the antecedent `fast-kan` [Li24] library was considered, technical challenges rendered this approach impractical. The investigation, therefore, led to the conclusion that a direct mitigation strategy was required. This motivated the implementation of gradient clipping [PMB13] as a necessary safeguard to ensure stable and reliable

[12] `LightAttentionFastKAN_1b1938a7-bf08-40ee-b53e-7c740e320b64`

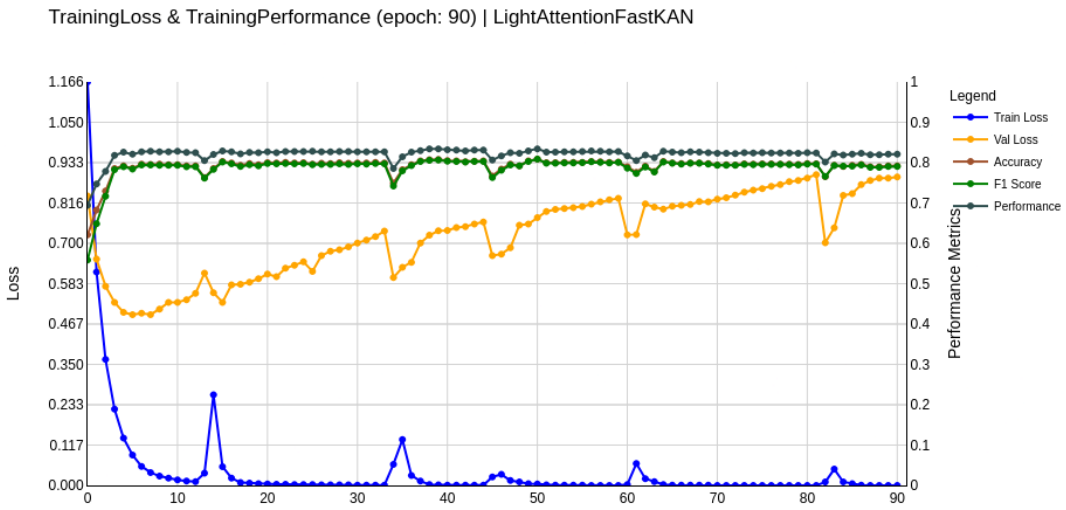


Figure 3.13.: Training and validation metrics for a `LightAttentionFastKAN` model prior to stabilization. The training loss (blue curve) exhibits abrupt spikes, which are characteristic of exploding gradients.

training, a practice that was adopted for all subsequent model refinements, as detailed in Section 2.8.2.

On the Omission of Pruning and the Challenge of Architectural Minimization

A key observation during this study was the divergence between training and validation loss, where the latter would sometimes increase after a certain number of epochs even as other metrics remained stable. This is a characteristic signature of overfitting, suggesting the models possessed a higher parameter capacity than was optimal for the task. This finding corroborates the principles outlined in the original KAN methodology [Liu+24], which proposes a powerful paradigm for discovering minimal, high-performance architectures. The authors demonstrate that smaller, more parsimonious KANs can achieve superior generalization, and they provide automated pruning mechanisms to identify and remove superfluous network components.

Despite its clear theoretical appeal for addressing the observed overfitting, this pruning technique was deliberately not incorporated into the refinement process due to several critical obstacles. The primary limitation was that the `faster-kan` [Del24] library—chosen for the computational efficiency crucial to exploring the hyperparameter space—does not possess a built-in pruning API. This represents a direct trade-off made between optimization speed and the availability of architectural simplification tools.

In the interest of due diligence, an investigation into the official `pykan` [Liu+24] library, which does implement pruning, was conducted. However, this exploration revealed significant practical and methodological issues. The library was found to have stability problems, particularly concerning the interaction between pruning and visualization. For instance, pruning nodes could result in ill-defined matrix dimensions for subsequent computations, leading to fatal errors in the underlying linear algebra solvers. Resolving these issues required manual source code patches merely to conduct basic tests.

More fundamentally, migrating the experimental framework to an entirely different KAN implementation would compromise the study's methodological consistency. Without any guaranty of performance parity between the `pykan` [Liu+24] and `faster-kan` [Del24] libraries under identical hyperparameters, any observed effects could not be confidently attributed to pruning alone. Such a change would introduce a significant confounding variable, invalidating comparisons with the results already gathered from the architectures under investigation.

Consequently, to preserve the integrity of the comparative analysis, the strategy of post-hoc pruning was set aside. The refinement process remained focused on regularization techniques that could be applied consistently within the established `faster-kan` [Del24] framework. The absence of automated pruning, however, implies that identifying a truly minimal architecture becomes a manual, iterative process. Therefore, while the trained models demonstrate strong performance, it is plausible that they are not yet in their most optimal, generalizable form. A key direction for future work would be a systematic, manual exploration of smaller network architectures to identify a configuration that further mitigates overfitting and potentially pushes the performance benchmarks for this task even higher.

3.4.3. Synergistic Effects and the Final Model Configurations

The preceding analytical process culminated in the final model configurations, whose superior performance was presented in Section 3.1. The journey from the initial hyperparameter candidates to these refined models was not the result of a single intervention, but a multi-faceted process combining standardization, targeted architectural refinement, and KAN-specific regularization. Table 3.1 provides a comprehensive summary of these modifications.

The first step in the refinement was to establish a fair basis for comparison. For models in the `ReducedFFN` family, the `reduced_seq_len` was standardized to 10 across all architectures. Furthermore, to enable a more direct comparison between the KAN and MLP variants of the same reduction strategy, the `reduced_channels` parameter was harmonized for the `UNetLayer` and `PositionalWeightedConvLayer` pairs.

`AttentionFastKAN`

The `AttentionFastKAN` model exhibited the most significant performance gains, ultimately becoming the best-performing architecture. A key driver of this success was the application of an aggressive "information bottleneck" to the FFN classifier, a principle inspired by the original `LightAttention` architecture [Stä+21]. This refinement, in synergy with narrowing the KAN grid window and adopting the two-stage attention architecture, culminated in a substantial 6.3 percentage point increase in accuracy from its initial configuration.

`AttentionMLP`

In contrast to its KAN counterpart, attempts to further simplify the FFN classifier head of the `AttentionMLP` did not yield appreciable performance improvements, suggesting the hyperparameter optimization had already identified a near-optimal configuration. The final 1.0 percentage point increase in accuracy is therefore primarily attributed to the architectural shift to the more effective two-stage attention-based reduction layer.

UNetFastKAN

The UNetFastKAN architecture presented a unique refinement challenge. Following the global standardization of the learning rate decay to 0.98 (see Section 3.4.2) and the implementation of gradient clipping (see Section 3.4.2), this model experienced a performance regression from its initial HPO-derived accuracy of 63.7% down to 62.7%.[13] Consequently, further analytical refinements to the KAN parameters—such as increasing the grid resolution and narrowing its range—were required to recover this performance, coincidentally bringing the final accuracy back to its original level. Separately, the decision to reduce the hidden layer count from three to two was influenced by the strong performance of its two-layer MLP counterpart and a mathematical consideration: with a relative layer reduction of 0.16, a three-layer network was found to produce a final hidden representation of just 39 nodes ($960 \times 10 \times 0.16^3 \approx 39$), a dimension close to the final output layer of 10 nodes, indicating diminishing returns for the third layer.

UNetMLP

The UNetMLP architecture exhibited a lower overall performance ceiling compared to the attention-based models. In line with this, its initial configuration derived from hyperparameter optimization proved to be robust; exploratory adjustments to the FFN classifier's size and depth yielded no significant performance gains, suggesting the model was already operating near its peak capacity for this framework. Given this combination of a lower performance ceiling and insensitivity to minor refinements, the architecture was deprioritized for further investigations.

PositionalFastKAN

For this novel architecture, the initial hidden layer configuration suggested by HPO proved robust, as exploratory adjustments to its depth and width did not yield positive results. The only beneficial refinement observed was a slight decrease in the KAN's grid resolution (num_grids).

However, the superior performance of its MLP counterpart suggests a potential architectural mismatch between the KAN-based classifier and the feature representation produced by the positional gating mechanism. It is hypothesized that this is due to the unique, high-dynamic-range statistical properties of the activations from the upstream PositionalWeightedConvLayer (as detailed in Section 2.6.1). While narrowing the KAN grid window (grid_diff) typically serves as a powerful regularizer, its effectiveness here is likely limited because it would prematurely clip the meaningful, high-magnitude signals that the positional layer was specifically engineered to produce.

Therefore, a more promising avenue for future work would involve redesigning the positional gating layer itself to produce activations within a more constrained range. This would create a synergistic relationship with the KAN classifier, allowing it to be regularized effectively without losing critical information from the reduction layer. The current results suggest that without such co-design, an MLP head may be better suited to process the "un-squashed" feature space from this specific reduction architecture.

[13] UNetFastKAN_c09fe1d7-af3c-4fbb-8f2c-02c1ff4f6330

`PositionalMLP`

The `PositionalMLP` emerged as a surprisingly strong performer, competitive with both the other custom architectures and the re-implemented baseline. As with its KAN variant, the initial FFN configuration proved to be well-calibrated. The notable performance gain is difficult to attribute to a single factor but is likely a result of several contributing elements. The architecture may have benefited significantly from the introduction of gradient clipping, as this model class was observed to suffer from gradient instability during initial trials. Furthermore, it is plausible that the condensed feature representation produced by the positional gating layer is inherently more amenable to processing by a traditional MLP. This architecture represents a promising avenue for future investigation.

`LightAttentionFastKAN`

The parameters for the `LightAttentionFastKAN` were primarily adjusted to establish a direct and equitable comparison with the re-implemented baseline. This necessitated deviating from the initial HPO-suggested FFN, which was relatively deep and wide, in favor of matching the baseline's single hidden layer of 32 nodes. This move was met with initial caution, as such a drastic dimensionality reduction from the 2048-node input was presumed to create an overly restrictive information bottleneck. This assumption prompted an intermediary experiment with a wider 128-node hidden layer; however, this yielded a surprisingly poor performance (59.3%).[14] Conversely, adopting the 32-node architecture—in conjunction with an increase in the KAN grid resolution (`num_grids` from 5 to 7)—dramatically increased accuracy to 65.6%.[15]

This finding is particularly noteworthy. The fact that further refinements to the KAN grid ultimately pushed performance even higher (to 66.1%), as detailed in Section 3.4.2, demonstrates a key insight. The regularization inherent in the KAN architecture is not merely an alternative to well-regularized MLP networks, like the one used in `LightAttention` (see Section 2.7.1), but can act as a powerful and complementary mechanism, yielding superior performance.

Notes on the Re-Implemented `LightAttention` Baseline

For the faithfully re-implemented baseline model, no changes were made to the core hyperparameters discovered by the original authors. The slight decrease in performance compared to their reported results can be plausibly attributed to the different training regime employed in this work, which incorporated a modest learning rate decay and gradient clipping to ensure stability across all compared architectures.

[14] `LightAttentionFastKAN_d42de715-75c9-415c-a452-3839b9287b0f`
[15] `LightAttentionFastKAN_0dd9e281-a0a6-4c53-8113-fb3a2449fa13`.

Synthesis of Findings and Architectural Insights

In summary, the refinement process was a multi-objective optimization balancing performance, stability, and architectural consistency. The final success was not the result of a single intervention, but rather the outcome of a synergy between multiple strategies: architectural regularization via information bottlenecks, hypothesis-driven parameter tuning, robust training protocols like gradient clipping, and a well-calibrated early stopping mechanism.

The results particularly underscore the core trade-off of KANs: their high expressivity enables remarkable learning speed but also renders them inherently prone to overfitting if not carefully managed. A key contribution of this work is the demonstration that the KAN-specific grid parameters (`grid_diff` and `num_grids`) serve as a powerful regularization lever to control this behaviour. It is critical, however, to contextualize these findings by noting that the MLP architectures employed in this study served as intentionally simple, unregularized baselines. Their purpose was to act as a clear point of reference, not to represent the state-of-the-art in MLP design. Consequently, this study does not—and cannot—make a categorical claim about the superiority of KANs over highly optimized MLP architectures.

Ultimately, the analysis reveals that optimal performance is achieved not by treating components as independent modules, but by co-designing the reduction layer and the classifier head. The strong synergistic relationship between the feature representation produced by the upstream layer and the architectural paradigm of the classifier underscores that a principled, diagnostic approach is required to harness the full potential of these powerful new architectures.

Model Architecture	Attribute	Initial Value	Final Value
`AttentionFastKAN`	Hidden Layers	2 | 0.14	1 | 0.04
	`reduced_seq_len`	15	10
	`grid_diff`	2.0	1.5
	`num_grids`	7	7
	accuracy	61.4%	67.6%
`AttentionMLP`	Hidden Layers	3 | 0.4	3 | 0.4
	`reduced_seq_len`	20	10
	accuracy	63.1%	64.1%
`UNetFastKAN`	Hidden Layers	3 | 0.16	2 | 0.16
	`reduced_seq_len`	10	10
	`reduced_channels`	960	960
	`grid_diff`	1.5	1.0
	`num_grids`	7	8
	accuracy	63.7%	63.7%
`UNetMLP`	Hidden Layers	2 | 0.8	2 | 0.8
	`reduced_seq_len`	20	10
	`reduced_channels`	512	960
	accuracy	62.9%	62.6%
`PositionalFastKAN`	Hidden Layers	1 | 0.14	1 | 0.14
	`reduced_seq_len`	10	10
	`reduced_channels`	576	512
	`grid_diff`	2.0	2.0
	`num_grids`	7	6
	accuracy	60.6%	61.1%
`PositionalMLP`	Hidden Layers	2 | 0.5	2 | 0.5
	`reduced_seq_len`	20	10
	`reduced_channels`	640	512
	accuracy	62.2%	64.4%
`LightAttentionFastKAN`	Hidden Layers	3 | 0.5	Exact ([32])
	`kernel_size`	5	9
	`conv_dropout`	0.48	0.25
	`grid_diff`	1.5	1.0
	`num_grids`	7	10
	accuracy	63.7%	66.1%
`LightAttention`	Hidden Layers	/	Exact ([32])
	`kernel_size`	/	9
	`conv_dropout`	/	0.25
	`ffn_conv_dropout`	/	0.25
	accuracy	65.7%	64.4%

Table 3.1.: Summary of parameter changes from initial candidates to final refined models.
Blue parameter: refined during the analytical phase.
Orange parameter: standardized for comparability across architectures.
Violet parameter: adjusted for comparability between KAN and MLP variants.
Green accuracy: improved ($\geq 1\%$). Red accuracy: decreased ($\geq 1\%$).

4. Conclusion

This thesis embarked on an inquiry into the practical application of Kolmogorov-Arnold Networks, moving the discourse from their compelling theoretical foundations to the more complex terrain of empirical application. The central question was not merely whether KANs could outperform traditional networks, but to understand the fundamental character of this emergent architecture within the context of a real-world scientific problem, exploring both their promises and the practical challenges of their implementation.

The research journey began with a broad, automated exploration of the parameter space, which yielded an initial cohort of promising models. However, this initial promise was shadowed by a significant generalization gap, a critical observation that prompted a shift from automated search to a principled, diagnostic process of refinement. Through a series of systematic investigations, specific weaknesses were identified, from training instability to overfitting, and addressed through a synergy of targeted interventions. These included architectural regularization via information bottlenecks, the implementation of stabilization safeguards such as gradient clipping, and the discovery of a potent, KAN-specific regularization lever in the manipulation of the B-spline grid parameters.

This investigation concludes not with a simple verdict on the superiority of KANs, but with a more complex and perhaps, more valuable set of insights. Instead, the finding is a nuanced portrait of the KAN architecture itself. The results strongly suggest that the high expressivity of KANs is a double-edged sword: it grants them a remarkable learning efficiency, often allowing them to converge faster than MLPs; however, this very same plasticity renders them acutely prone to overfitting the idiosyncrasies of the training data. Addressing this requires a more deliberate and nuanced approach to regularization than their MLP counterparts might demand. The success of the top-performing model must therefore be viewed with a degree of epistemological humility; it is a promising result, but one that underscores the challenge of separating true architectural merit from the fortunate contingencies of a single training run, particularly in the context of a finely-tuned early-stopping protocol.

These findings resonate with the current landscape of the field, where KANs remain conspicuously absent from state-of-the-art models. The significant overhead in stabilizing and regularizing these networks, compounded by an underdeveloped and fragmented software ecosystem, suggests that the paradigm has not yet reached a state of practical maturity.[1] The faster convergence they offer is coupled with the non-trivial challenge of arresting their trajectory before they over-specialize.

[1] As of late 2025, the KAN software landscape is marked by several independent and unmaintained libraries. For instance, the `faster-kan` [Del24] and `fast-kan` [Li24] repositories saw their last commits in mid-2024, while even the official `pykan` [Liu+24] has received only sporadic updates.

Ultimately, this thesis contributes not a definitive answer, but a clearer articulation of the question. It reframes the inquiry from "Are KANs a good alternative to MLP?" to "What are the conditions under which KANs can be made robust and reliable?" The value of this work lies in transforming an unknown territory into a structured problem, identifying the distinct challenges of instability, expressivity, and regularization. While KANs may not yet be a ready replacement for the venerable MLP, this study affirms that they are a rich subject of inquiry, offering unique mechanisms for control and compelling hints of greater parsimony. The path forward is now better defined, not by a single, triumphant solution, but by the patient, principled investigation of these fundamental architectural tensions.

A. Code and Data Availability

For the purpose of reproducibility and further inquiry, all source code, analysis scripts, model configuration files, and supplementary data associated with this thesis are made openly available in a digital repository.

The main project repository can be accessed at:

`https://github.com/JinMaxx/Protein-Localization-using-KANs/tree/main`

For an interactive demonstration and a way to quickly get started with the core concepts of this work, a Google Colaboratory notebook is provided:

`https://colab.research.google.com/github/JinMaxx/Protein-Localization-using-KANs/blob/main/colab_notebook.ipynb`

The repository is structured to provide easy access to key components of this research:

Model Reports
The detailed reports for the models, including their complete configurations, performance metrics, and evaluation plots, can be found in the following directory:

`/data/reports/`

Hyperparameter Optimization Data
The raw data and aggregated results from the Optuna hyperparameter study, (Section 2.8.4), are located at:

`/data/hyper_param/`

Source Code
All custom Python modules for model architectures, data processing, and analysis are located within the main source directory of the repository.

`/source/`

List of Figures

3.1. Final model accuracy (mean macro-averaged) comparison on the `setHARD` test set. Error bars represent the 95% confidence interval calculated from 100 bootstrap iterations. Models utilizing a KAN-based classifier are shown in orange, with the original ProtT5-LA (LightAttention) baseline model in gray. The `AttentionFastKAN` model achieved the highest mean accuracy. . 19

3.2. Training and validation curves for the re-implemented `LightAttention` model. Peak performance on the validation set is achieved at epoch 36, after which the validation loss remains stable. 20

3.3. Training and validation curves for the `AttentionFastKAN` model. Note the early rise in validation loss from epoch 3, while peak performance on the validation set is achieved later at epoch 13. 21

3.4. Normalized confusion matrix for the `LightAttention` model on the `setHARD` test set. 22

3.5. Normalized confusion matrix for the `AttentionFastKAN` model on the `setHARD` test set. 23

3.6. Prediction confidence distribution for the `LightAttention` model. The model displays a reserved confidence profile, with broad distributions for most classes. 24

3.7. Prediction confidence distribution for the `AttentionFastKAN` model. The model exhibits a more polarized and decisive profile compared to the baseline. 24

3.8. Distribution of the 10 subcellular localization classes across the combined training, validation, and test datasets. The significant class imbalance is evident, with **peroxisome** being the rarest class at 1.1% and **nucleus** the most common at 28.5%. 26

3.9. Log-scaled histogram of protein sequence lengths across the aggregated dataset. The prominent spike at 1023 residues corresponds to the effective truncation limit after excluding the end-of-sequence token from the `ProtT5` embedding model. The distribution justifies the selection of 1023 as the uniform sequence length to minimize information loss. 27

3.10. Average **weighted validation accuracy** comparison of different reduction layer architectures from the hyperparameter search. Bars represent the mean accuracy across all successful trials, with error bars indicating the standard deviation. To better distinguish the final feed-forward network, `FastKAN`-based models are rendered with partial transparency, while `MLP`-based models are solid. The `N` value above each bar indicates the number of successful trials included in the average; this number varies due to the automated pruning of failed or prematurely converged trials during the optimization process. The percentage values below show the concrete accuracy score. 28

3.11. Macro F1-Score versus model memory footprint (in MB) for all successful hyperparameter trials, faceted by base architecture. Each point represents a single trial, colored by its FFN type (`FastKAN`or `MLP`). These plots were used to identify the point of diminishing returns for model size. 29
3.12. Initial model accuracies on the `setHARD` test set. These first-generation candidates were selected based on hyperparameter optimization against the validation set. 31
3.13. Training and validation metrics for a `LightAttentionFastKAN` model prior to stabilization. The training loss (blue curve) exhibits abrupt spikes, which are characteristic of exploding gradients. 36

List of Tables

2.1. Dataset partitions for training, validation, and testing [Stä+21]. 5
2.2. Consolidation of granular subcellular annotations into 10 canonical classes. 6

3.1. Summary of parameter changes from initial candidates to final refined models.
Blue parameter: refined during the analytical phase.
Orange parameter: standardized for comparability across architectures.
Violet parameter: adjusted for comparability between KAN and MLP variants.
Green accuracy: improved ($\geq 1\%$). Red accuracy: decreased ($\geq 1\%$). . . . 41

Bibliography

[Aki+19] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[Alb+22] Bruce Alberts et al. *Molecular Biology of the Cell*. 7th. New York, NY: W. W. Norton & Company, 2022. ISBN: 978-0-393-68031-7.

[Alm+17] José Juan Almagro Armenteros et al. "DeepLoc: prediction of protein subcellular localization using deep learning". In: *Bioinformatics* 33.21 (July 2017), pp. 3387–3395. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btx431. eprint: https://academic.oup.com/bioinformatics/article-pdf/33/21/3387/50315453/bioinformatics_33_21_3387.pdf. URL: https://doi.org/10.1093/bioinformatics/btx431.

[Arn09] V. I. Arnold. "On the representation of functions of several variables as a superposition of functions of a smaller number of variables". In: *Collected Works: Representations of Functions, Celestial Mechanics, and KAM Theory, 1957–1965*. Ed. by Alexander B. Givental et al. English translation of the original paper published in: *Matematicheskoe Prosveshchenie*, Ser. 2, Vol. 3, pp. 41–61 (1958). Berlin, Heidelberg: Springer, 2009, pp. 25–46. DOI: 10.1007/978-3-642-01742-1_5.

[BCB14] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate". In: *arXiv preprint arXiv:1409.0473* (2014).

[Ber+18] Jeremy M. Berg et al. *Stryer Biochemie*. 8th ed. Translated by Andreas Held, Gudrun Maxam, Lothar Seidler, Bärbel Häcker, Birgit Jarosch. Berlin, Heidelberg: Springer Spektrum, 2018. ISBN: 978-3-662-54619-2. DOI: 10.1007/978-3-662-54620-8.

[BG09] Jürgen Braun and Michael Griebel. "On a constructive proof of Kolmogorov's superposition theorem". In: *Constructive Approximation* 30.3 (2009), pp. 653–675. ISSN: 1432-0940. DOI: 10.1007/s00365-009-9054-2.

[BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[Boo78] Carl de Boor. *A Practical Guide to Splines*. 1st. Vol. 27. Applied Mathematical Sciences. New York, NY: Springer-Verlag, 1978. ISBN: 978-0-387-95366-3.

[Buh00] Martin D. Buhmann. "Radial basis functions". In: *Acta Numerica* 9 (2000), pp. 1–38. DOI: 10.1017/S0962492900000015.

[Che66] E. W. Cheney. *Introduction to Approximation Theory*. New York: McGraw-Hill, 1966.

[Del24] Athanasios Delis. *FasterKAN*. https://github.com/AthanasiosDelis/faster-kan/. 2024.

[Dev+19] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[DG06] Jesse Davis and Mark Goadrich. “The relationship between Precision-Recall and ROC curves”. In: *Proceedings of the 23rd International Conference on Machine Learning*. ICML ’06. Pittsburgh, Pennsylvania, USA: Association for Computing Machinery, 2006, pp. 233–240. ISBN: 1595933832. DOI: 10.1145/1143844.1143874. URL: https://doi.org/10.1145/1143844.1143874.

[DV16] Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”. In: *arXiv preprint arXiv:1603.07285* (2016).

[Efr79] Bradley Efron. “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7 (Jan. 1979), pp. 1–26.

[Eln+21] Ahmed Elnaggar et al. “ProtTrans: Towards Cracking the Language of Lifes Code Through Self-Supervised Deep Learning and High Performance Computing”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021), pp. 1–1. DOI: 10.1109/TPAMI.2021.3095381.

[FM02] R. Fry and S. McManus. “Smooth bump functions and the geometry of banach spaces: A brief survey”. In: *Expositiones Mathematicae* 20.2 (2002), pp. 143–183. ISSN: 0723-0869. DOI: https://doi.org/10.1016/S0723-0869(02)80017-2. URL: https://www.sciencedirect.com/science/article/pii/S0723086902800172.

[GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[Goo25] Google. *Google Colaboratory*. 2025. URL: https://colab.research.google.com (visited on 10/10/2025).

[He+16] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[HG09] Haibo He and Edwardo A. Garcia. “Learning from Imbalanced Data”. In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), pp. 1263–1284. DOI: 10.1109/TKDE.2008.239.

[HN98] Jerry L Hintze and Ray D Nelson. “Violin plots: a box plot-density trace synergism”. In: *The American Statistician* 52.2 (1998), pp. 181–184.

[HTF09] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer Series in Statistics. New York, NY: Springer, 2009. ISBN: 978-0-387-84857-0. DOI: 10.1007/978-0-387-84858-7. URL: https://doi.org/10.1007/978-0-387-84858-7.

[IS15] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. pmlr. 2015, pp. 448–456.

[Kol57] A. N. Kolmogorov. "On the representation of continuous functions of several variables by superpositions of continuous functions of one variable and addition". In: *Doklady Akademii Nauk SSSR* 114.5 (1957). (In Russian) For the English translation, see *Amer. Math. Soc. Transl. (2)*, 28 (1963), pp. 55–59., pp. 953–956. URL: https://www.mathnet.ru/eng/dan22050.

[Lan+07] A. Lange et al. "Classical nuclear localization signals: definition, function, and interaction with importin alpha". In: *The Journal of Biological Chemistry* 282.8 (2007), pp. 5101–5105. DOI: 10.1074/jbc.R600026200.

[LB88] David Lowe and D Broomhead. "Multivariable functional interpolation and adaptive networks". In: *Complex systems* 2.3 (1988), pp. 321–355.

[LeC+02] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (2002), pp. 2278–2324.

[Lee+19] Juho Lee et al. "Set transformer: A framework for attention-based permutation-invariant neural networks". In: *International conference on machine learning*. PMLR. 2019, pp. 3744–3753.

[Les+93] Moshe Leshno et al. "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function". In: *Neural networks* 6.6 (1993), pp. 861–867.

[LH17] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: *arXiv preprint arXiv:1711.05101* (2017).

[Li24] Ziyao Li. "Kolmogorov-Arnold Networks are Radial Basis Function Networks". In: *arXiv preprint arXiv:2405.06721* (May 2024). DOI: 10.48550/arXiv.2405.06721. arXiv: 2405.06721 [cs.LG]. URL: https://arxiv.org/abs/2405.06721.

[Liu+24] Ziming Liu et al. "KAN: Kolmogorov-Arnold Networks". In: *arXiv preprint arXiv:2404.19756* (2024).

[Mic+17] Paulius Micikevicius et al. "Mixed precision training". In: *arXiv preprint arXiv:1710.03740* (2017).

[NH10] Vinod Nair and Geoffrey E Hinton. "Rectified linear units improve restricted boltzmann machines". In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.

[NW22] Kenta Nakai and Liping Wei. "Recent Advances in the Prediction of Subcellular Localization of Proteins and Related Topics". In: *Frontiers in Bioinformatics* 2 (May 2022), p. 910531. ISSN: 2673-7647. DOI: 10.3389/fbinf.2022.910531.

[Pas+19] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[PMB13] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks". In: *International conference on machine learning*. Pmlr. 2013, pp. 1310–1318.

[Pre02] Lutz Prechelt. "Early stopping-but when?" In: *Neural Networks: Tricks of the trade*. Springer, 2002, pp. 55–69.

[RAH04] José Antonio Rodriguez, Wendy W.Y. Au, and Beric R. Henderson. "Cytoplasmic mislocalization of BRCA1 caused by cancer-associated mutations in the BRCT domain". In: *Experimental Cell Research* 293.1 (2004), pp. 14–21. ISSN: 0014-4827. DOI: https://doi.org/10.1016/j.yexcr.2003.09.027. URL: https://www.sciencedirect.com/science/article/pii/S0014482703005445.

[RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation". In: *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[See04] Matthias Seeger. "Gaussian processes for machine learning". In: *International journal of neural systems* 14.02 (2004), pp. 69–106.

[Sen+24] Tobias Senoner et al. *ProtSpace: Click-Through Example for Generating Embeddings*. https://colab.research.google.com/github/tsenoner/protspace/blob/main/examples/notebook/ClickThrough_GenerateEmbeddings.ipynb. Accessed: 2025-08-17. This notebook is part of the ProtSpace tool, formally described in [Sen+25]. 2024.

[Sen+25] Tobias Senoner et al. "ProtSpace: A Tool for Visualizing Protein Space". In: *Journal of Molecular Biology* (2025), p. 168940. ISSN: 0022-2836. DOI: https://doi.org/10.1016/j.jmb.2025.168940. URL: https://www.sciencedirect.com/science/article/pii/S0022283625000063.

[Stä+21] Hannes Stärk et al. "Light attention predicts protein location from the language of life". In: *Bioinformatics Advances* 1.1 (Nov. 2021), vbab035. DOI: 10.1093/bioadv/vbab035. URL: https://doi.org/10.1093/bioadv/vbab035.

[The] The HDF Group. *Hierarchical Data Format, Version 5*. https://www.hdfgroup.org/solutions/hdf5/. Accessed: 2025-10-08.

[The17] The UniProt Consortium. "UniProt: the universal protein knowledgebase". In: *Nucleic Acids Research* 45.D1 (Jan. 2017). Erratum in: Nucleic Acids Res. 2018 Mar 16;46(5):2699, pp. D158–D169. DOI: 10.1093/nar/gkw1099.

[The21] The UniProt Consortium. "UniProt: the universal protein knowledgebase in 2021". In: *Nucleic Acids Research* 49.D1 (Jan. 2021), pp. D480–D489. DOI: 10.1093/nar/gkaa1100.

[Vas+17] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.